



UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Análise de LLMs baseados LLaMA 3 para a Verificação Automática de Normas de Arquitetura, Engenharia e Construção

Proposta de Dissertação de Mestrado

Eike Natan Sousa Brito



São Cristóvão – Sergipe

2026

UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Eike Natan Sousa Brito

**Análise de LLMs baseados LLaMA 3 para a Verificação
Automática de Normas de Arquitetura, Engenharia e
Construção**

Proposta de Dissertação de Mestrado apresentada ao Programa de Pós-Graduação em Ciência da Computação da Universidade Federal de Sergipe como requisito parcial para a obtenção do título de mestre em Ciência da Computação.

Orientador(a): Andre Britto de Carvalho

São Cristóvão – Sergipe

2026

Resumo

A adoção do BIM (Building Information Modeling) está transformando digitalmente a construção civil ao centralizar informações de todo o ciclo de vida de um projeto, aumentando a precisão, reduzindo erros e otimizando recursos. Contudo, o crescimento do uso do BIM traz desafios, como o gerenciamento de grandes volumes de dados e a integração de normas regulatórias complexas. À medida que os projetos baseados em BIM se tornam mais comuns, observa-se um aumento expressivo no volume de dados gerados, o que exige soluções tecnológicas avançadas. A aplicação de Inteligência Artificial (IA), especialmente por meio de Aprendizado de Máquina (AM), tem contribuído significativamente para a automação de processos no setor. Por exemplo, modelos de IA têm sido utilizados para detecção automática de objetos em fotos ou até melhoria no processo de montagem na construção por meio de regras de design. Contudo, no contexto das normas regulatórias de Arquitetura, Engenharia e Construção (AEC), ainda existem lacunas na integração dessas tecnologias em níveis nacionais e internacionais devido à complexidade semântica e à falta de padronização desses documentos. Uma solução promissora é a metodologia RASE, que transforma normas regulatórias em dados computáveis, permitindo cálculos automáticos e validações em projetos reais. Modelos de linguagem de grande escala (LLMs), como o ChatGPT e o Llama 3, têm sido explorados para interpretar documentos regulatórios, facilitando a análise semântica e a extração de informações relevantes. No entanto, essas abordagens enfrentam limitações, como a dificuldade de interpretar documentos com linguagem altamente técnica e a necessidade de treinamento específico com dados do setor AEC, o que pode restringir sua aplicabilidade em contextos mais amplos. Esta pesquisa propõe a criação de três LLMs baseados no LLaMa 3 para compreender e transformar normas em regras estruturadas voltadas para o setor AEC. Os três modelos específicos serão desenvolvidos e treinados com abordagens distintas para converter normas em padrões RASE, resultando em dados verificáveis no software Revit. Serão utilizadas técnicas como Engenharia de Prompt, Fine-Tuning e Retrieval-Augmented Generation (RAG) para aprimorar o desempenho dos modelos. A validação envolverá a aplicação dos LLMs em projetos reais no Revit, identificando a conformidade regulatória por meio de códigos de cores.

Palavras-chave: Aprendizado de Máquina, Processamento de Linguagem Natural, LLM, LLaMa, BIM, RASE.

Abstract

The adoption of Building Information Modeling (BIM) is digitally transforming the construction industry by centralizing information throughout a project's entire lifecycle, increasing accuracy, reducing errors, and optimizing resources. However, the growing use of BIM also presents challenges, such as managing large volumes of data and integrating complex regulatory standards. As BIM-based projects become more common, there is a significant increase in data generation, requiring advanced technological solutions. The application of Artificial Intelligence (AI), particularly through Machine Learning (ML), has significantly contributed to automating processes in the industry. For instance, AI models have been used for automatic object detection in images or even improving assembly processes in construction through design rules. However, in the context of regulatory standards in Architecture, Engineering, and Construction (AEC), gaps still exist in the integration of these technologies at national and international levels due to the semantic complexity and lack of standardization of these documents. A promising solution is the RASE methodology, which converts regulatory standards into computable data, enabling automatic calculations and validations in real projects. Large Language Models (LLMs), such as ChatGPT and Llama 3, have been explored to interpret regulatory documents, facilitating semantic analysis and the extraction of relevant information. However, these approaches face limitations, such as difficulties in interpreting documents with highly technical language and the need for specific training with AEC sector data, which can restrict their broader applicability. This research proposes developing three LLMs based on LLaMa 3 to understand and transform standards into structured rules tailored for the AEC sector. The three specific models will be developed and trained using distinct approaches to convert standards into RASE patterns, resulting in verifiable data within Revit software. Techniques such as Prompt Engineering, Fine-Tuning, and Retrieval-Augmented Generation (RAG) will be employed to enhance the models' performance. Validation will involve applying the LLMs to real projects in Revit, identifying regulatory compliance through color-coded visualizations.

Keywords: Machine Learning, Natural Language Processing, LLM, LLaMa, BIM, RASE.

Lista de ilustrações

Figura 1 – Representação do BIM - Fonte: HOCH	15
Figura 2 – Matriz de confusão.	17
Figura 3 – Exemplo da transformação de palavras em <i>word embeddings</i>	21
Figura 4 – Exemplo da transformação de <i>word embeddings</i> para <i>sentence embeddings</i>	22
Figura 5 – Rede Neural MLP.	23
Figura 6 – Estrutura de um neurônio no MLP.	24
Figura 7 – Exemplo do deslocamento do gráfico através da mudança da constante bias.	24
Figura 8 – Tipos de camadas e conexões de nodos.	28
Figura 9 – Exemplo de um modelo seq2seq para tradução do inglês para português.	31
Figura 10 – Exemplo de um modelo com o mecanismo de atenção para tradução do inglês para português.	33
Figura 11 – Exemplo do cálculo de uma camada de <i>self-attention</i>	34
Figura 12 – Representação da pilha (<i>stack</i>) do <i>transformer</i>	35
Figura 13 – Camadas do <i>encoder</i> e <i>decoder</i> do <i>transformer</i>	36
Figura 14 – Arquitetura RAG: Geração Aumentada por Recuperação	40
Figura 15 – Utilização da metodologia RASE na NBR 9050	43
Figura 16 – Pipeline da pesquisa.	53
Figura 17 – Cronograma de execução	57

Lista de abreviaturas e siglas

DCOMP	Departamento de Computação
UFS	Universidade Federal de Sergipe
BIM	Building Information Modeling ou Modelagem da Informação da Construção
AEC	Architecture, Engineering and Construction ou Arquitetura, Engenharia e Construção
AM	Machine Learning ou Aprendizado de Máquina
IA	Artificial Intelligence ou Inteligência Artificial
PLN	Natural Language Processing ou Processamento de Linguagem Natural
IAG	Generative AI ou IA Generativa
FT	Fine-Tuning ou Ajuste Fino
TL	Transfer Learning ou Aprendizado por Transferência
RAG	Retrieval-Augmented Generation ou Geração Aumentada por Recuperação
LLM	Large Language Models ou Modelos de Linguagem de Grande Escala
EP	Prompt Engineering ou Engenharia de Prompt
ACC	Accuracy ou Acurácia
PRE	Precision ou Precisão
REC	Recall ou Revocação
F1	F1-Score ou Medida F1
RNA	Artificial Neural Networks ou Redes Neurais Artificiais
MLP	Multi-Layer Perceptron ou Perceptron Multi-Camadas
MSE	Mean Squared Error ou Erro Quadrático Médio
RMSE	Root Mean Squared Error ou Raiz do Erro Quadrático Médio
MAE	Mean Absolute Error ou Erro Médio Absoluto
RNN	Recurrent Neural Networks ou Redes Neurais Recorrentes

LSTM	Long Short-Term Memory ou Memória de Longo e Curto Prazo
BiLSTM	Bidirectional Long Short-Term Memory ou Memória de Longo e Curto Prazo Bidirecional
FLOP	Floating Point Operations ou Operações de Ponto Flutuante
SFT	Supervised Fine-Tuning ou Ajuste Fino Supervisionado
RM	Reward Model ou Modelo de Recompensa
VecDBs	Vector Databases ou Bases de Dados Vetoriais

Lista de símbolos

θ	Bias
w	Peso
u	Ponto de ativação
q	Query
k	Key
v	Value
W^q	Cabeça de atenção da Query
Q^k	Cabeça de atenção da Key
W^v	Cabeça de atenção do Value

Sumário

1	Introdução	9
1.1	Objetivos	11
1.2	Organização da Proposta	12
2	Fundamentação Teórica	13
2.1	BIM	13
2.2	Aprendizado de Máquina	15
2.3	PLN	18
2.4	Redes Neurais Artificiais	21
2.5	Memória Longa de Curto Prazo	29
2.6	Transformers	30
2.7	LLM	36
2.8	RASE	41
2.9	Trabalhos Relacionados	44
2.9.1	Análise dos Trabalhos Relacionados	45
3	Proposta	47
3.1	Materiais e Métodos	47
3.2	Desenvolvimento Inicial	53
3.3	Desafios Futuros	54
3.4	Fases da projeto e cronograma	55
	Referências	58

1

Introdução

A área de execução de projetos e fiscalização de obras sustenta-se cada vez mais por meio de tecnologias e metodologias que asseguram uma melhor aplicação dos recursos. Com a evolução tecnológica, tornou-se possível corrigir deficiências do setor e atender às exigências do mercado. Nesse contexto, muitas empresas de engenharia estão migrando de padrões bidimensionais (2D) para tridimensionais (3D). Essa transição introduz o conceito de *Building Information Modeling* (BIM), uma metodologia inovadora para os setores de Arquitetura, Engenharia e Construção (AEC) (EASTMAN; TEICHOLZ; SACKS, 2011).

O BIM (*Building Information Modeling*) é um processo que abrange o planejamento e a execução de construções à luz de modelos tridimensionais detalhados. A metodologia pode ser classificada em diversas dimensões, de 3D a 7D, cada uma representando um aspecto específico da utilização do BIM: produção do modelo ou projetos (3D); planejamento de tempo (4D); gestão de custos (5D); sustentabilidade do edifício (6D); e gerenciamento, manutenção e operação pós-construção (7D) (EASTMAN; TEICHOLZ; SACKS, 2011). É importante ressaltar que o BIM não é um software, mas sim um conceito fundamentado em sistemas informatizados como o CAD. Alguns dos softwares que implementam o conceito BIM são o Revit®, da Autodesk®, e o ArchiCAD®, da Graphisoft®.

O BIM permite criar modelos virtuais detalhados, facilitando a visualização e a detecção de interferências antes da construção física. As principais aplicações do BIM incluem análise de projetos, orçamento e planejamento, coordenação modular, gestão do ciclo de vida, visualização arquitetônica, entre outros (VARGAS, 2023a).

Com a adoção do BIM, a quantidade de dados gerados por projetos cresce significativamente, atraindo o interesse de cientistas de dados. Esses profissionais trabalham com grandes volumes de informações, coletando, analisando e interpretando dados para gerar lucros para as organizações (DAVENPORT; PATIL, 2012). Uma das áreas mais promissoras para cientistas de dados é o Aprendizado de Máquina (AM).

O Aprendizado de Máquina, uma subárea da Inteligência Artificial (IA), explora a criação de algoritmos capazes de compreender dados de forma autônoma (TAN; STEINBACH; KUMAR, 2006). Por meio do AM, é possível extrair e reconhecer padrões em grandes volumes de dados, construindo modelos que executam tarefas complexas, fazem previsões e reagem de maneira inteligente a diferentes situações.

No campo da Engenharia Civil, a IA tem transformado processos tradicionais em abordagens automatizadas, promovendo maior eficiência, precisão e sustentabilidade. Algoritmos de aprendizado de máquina, redes neurais e aprendizado profundo permitem que sistemas revisem e otimizem projetos desde a fase de planejamento até a execução de grandes obras (FUCHS et al., 2024).

O trabalho de Liu investigou o uso de Aprendizado de Máquina para melhorar o processo de montagem na construção por meio de regras de design. O autor concluiu que a combinação de BIM, AM e regras de design pode otimizar o processo de montagem, resultando em projetos mais eficientes e econômicos (LIU; ZHANG; LI, 2022). O autor Alawadhi treinou Redes Neurais Artificiais com dados BIM com o objetivo de reconhecer objetos de construção em fotos e concluiu que as imagens geradas por computador dos modelos BIM podem resultar em segmentação semântica eficaz em fotos reais de edifícios, indicando a viabilidade de usar dados sintéticos para treinar IA em problemas arquitetônicos do mundo real (ALAWADHI; YAN, 2023). O cientista Saka realizou um treinamento com dados BIM para prever a circularidade de resíduos de demolição de construção. A partir de 2.280 projetos de demolição, foi possível desenvolver um sistema capaz de prever as quantidades de materiais recicláveis e destinar os resíduos para aterros de demolição, facilitando um melhor planejamento e gestão nessa área (SAKA et al., 2024).

No entanto, o BIM não oferece uma solução direta para a gestão de normas regulatórias voltadas para o setor AEC, uma vez que essas normas são reguladas por região. Cada região possui suas próprias regras, muitas das quais são interpretativas, dificultando a automação desse processo (VARGAS, 2023b).

O Processamento de Linguagem Natural (PLN) possibilita que modelos de AM reconheçam padrões e processem informações, gerando resultados como sistemas de tradução, conversas e respostas automatizadas. Uma subárea emergente do PLN é a IA Generativa (IAG), cujos modelos são capazes de criar novos dados com base em seus *datasets* de treinamento. Esses modelos são amplamente usados na geração de textos, imagens e áudios. Exemplos de modelos generativos incluem o ChatGPT, da OpenAI (OPENAI et al., 2024), e o Llama 3, da Meta (AI@META, 2024). Esses algoritmos são projetados para produzir respostas contextualmente sensíveis em linguagem humana e são classificados como LLMs (INAN et al., 2023).

Os Modelos de Linguagem de Grande Escala (LLMs) são sistemas de inteligência artificial projetados para compreender e gerar linguagem humana de forma avançada. Esses modelos são treinados em vastas quantidades de dados textuais, permitindo-lhes capturar nuances linguísticas e contextuais. Arquiteturas como os transformadores são comumente utilizadas

nesses modelos devido à sua eficácia em processar sequências de texto (ZHAO et al., 2023).

Tecnologias relacionadas à LLMs têm transformado o acesso ao conhecimento e a interação com o mundo. Técnicas como *Fine-Tuning* (FT) (GUO et al., 2019), *Transfer Learning* (TL) (GUO et al., 2019) e *Retrieval-Augmented Generation* (RAG) (GAO et al., 2023) permitem incorporar novas funcionalidades a esses modelos, ampliando suas aplicações.

No setor de Arquitetura, Engenharia e Construção (AEC), os Modelos de Linguagem de Grande Escala (LLMs) podem auxiliar na interpretação de normas regulatórias e na conversão dessas informações em formatos processáveis por máquinas (DU; NOUSIAS; BORRMANN, 2024). Contudo, há lacunas na literatura quanto à integração da LLMs com tecnologias emergentes na indústria AEC. Uma dessas lacunas está na interpretação de leis, códigos e padrões regulatórios em níveis nacionais e internacionais, que historicamente apresentam desafios para a IA devido à complexidade semântica desses documentos (FUCHS et al., 2024).

Para abordar esse problema, Hjelseth desenvolveu a metodologia RASE, que transforma normas regulatórias em dados computáveis (HJELSETH; NISBET, 2011). Os dados computáveis na metodologia RASE são informações extraídas de normas regulatórias e estruturadas em quatro atributos: **Requisito**, que define o que deve ser atendido; **Aplicabilidade**, que indica onde a regra se aplica; **Seleção**, que apresenta opções para cumpri-la; e **Exceção**, que especifica casos em que a regra não se aplica. Essa organização permite que softwares BIM interpretem e verifiquem automaticamente a conformidade de projetos, facilitando ajustes e validações com maior eficiência e precisão (HJELSETH; NISBET, 2011).

Com a transformação das normas regulatórias para o formato RASE, torna-se possível desenvolver um software integrado ao Revit, capaz de validar cada elemento do projeto de acordo com as normas apresentadas (SANTOS; SAMPAIO, 2021). Contudo, a transformação das normas para o formato RASE é atualmente realizada de forma manual, exigindo que cada regra seja analisada e convertida individualmente. Além disso, dependendo do projeto BIM, pode haver centenas ou até milhares de elementos, o que gera um alto custo de tempo para realizar todo o processo de conversão e validação dos dados.

1.1 Objetivos

Esta pesquisa propõe o desenvolvimento de um modelo de linguagem LLM baseado no Llama 3 para interpretar normas regulatórias da AEC e convertê-las em dados estruturados no formato RASE. Três modelos serão desenvolvidos: **R1** para extração e separação de regras regulatórias, **R2** para organização dessas regras nos quatro atributos RASE (Requisito, Aplicabilidade, Seleção, e Exceção) e **R3** para converter as regras em dados diretamente aplicáveis em softwares BIM, como o Revit.

Quatro abordagens de treinamento serão avaliadas: Engenharia de Prompt (**EP**), Enge-

nharia de Prompt com Fine-Tuning (**EPFT**), Engenharia de Prompt com RAG (**EPRAG**), e Engenharia de Prompt com Fine-Tuning e RAG (**EPFTRAG**). A eficácia será medida em termos de precisão, eficiência e generalização na interpretação e aplicação das normas.

Os modelos treinados serão aplicados em projetos no Revit, com objetos destacados em cores específicas para indicar conformidade: verde para elementos conformes, vermelho para não conformes e azul para itens não identificados ou irrelevantes.

1.2 Organização da Proposta

Os capítulos seguintes estruturam esta pesquisa de forma detalhada e consistente. O segundo aborda os fundamentos teóricos e trabalhos relacionados, explorando conceitos como BIM, Inteligência Artificial, Processamento de Linguagem Natural, Aprendizado de Máquina, Redes Neurais (incluindo LSTM, BiLSTM e Transformers), LLM e técnicas específicas e por fim, o método RASE.

O terceiro capítulo é dedicado à proposta a ser utilizada, detalhando os passos e estratégias empregados para alcançar os objetivos gerais e específicos da pesquisa, além de apresentar o cronograma de execução da proposta.

2

Fundamentação Teórica

2.1 BIM

O BIM (*Building Information Modeling*, ou Modelagem da Informação da Construção) não é uma tecnologia específica, mas sim um processo integrado ao planejamento e execução de construções. De acordo com [Penttilä \(2016\)](#), o BIM consiste em um conjunto estruturado de políticas, processos e ferramentas que formam uma metodologia para gerenciar os dados digitais e os projetos de construção ao longo do ciclo de vida do edifício.

Esse recurso surgiu "após a inovação do CAD 3D paramétrico", tecnologia que foi amplamente adotada em setores como aeroespacial, automotivo e de manufatura ([MONTEIRO, 2011](#)). O BIM vai além de representações tridimensionais que incluem dimensões, volumes e áreas, incorporando informações adicionais, como acabamento, custos, fabricantes e relações com outros elementos. Isso permite extrair informações de maneira mais eficiente e, em caso de alterações estruturais, atualizar todos os dados do projeto de forma centralizada. Apesar de seu caráter inovador, a implementação do BIM demanda equipes bem preparadas, tanto na etapa de controle quanto na de acompanhamento, o que pode elevar o custo inicial, embora proporcione uma gestão mais eficiente durante todo o ciclo de vida da construção ([COELHO; NOVAES, 2018](#)).

Entre os principais benefícios do BIM está a possibilidade de trabalhar com várias dimensões em um único projeto. Com base nas informações adicionadas ao modelo, é possível reduzir falhas, otimizar prazos e garantir maior controle sobre as atualizações ([PARREIRA, 2013](#)).

Segundo ([EASTMAN; TEICHOLZ; SACKS, 2011](#)), o BIM possui sete dimensões que podem ser exploradas independentemente, desde que o modelo 3D já esteja desenvolvido. Isso significa, por exemplo, que uma análise 6D pode ser realizada sem a necessidade de uma inspeção 4D, como ilustrado na Figura 1.

- **BIM 3D - Modelo Virtual 3D:** Ampliando o conceito de modelagem 2D, o BIM 3D permite a visualização completa dos elementos em três dimensões (X, Y e Z), além de identificar colisões utilizando algoritmos originados da computação gráfica. Essa funcionalidade reduz significativamente custos na fase de design e na execução da obra.
- **BIM 4D - Programação:** Esta dimensão organiza o modelo 3D em etapas, atribuindo sequências de montagem aos elementos. Além de possibilitar a visualização das fases do projeto, ela permite simular cronogramas e planejar prazos de entrega com maior precisão.
- **BIM 5D - Estimativa de Custos:** Proporciona uma rápida estimativa de custos para diferentes fases do projeto, gerando relatórios financeiros e permitindo comparar alternativas em termos de tempo de execução e investimentos.
- **BIM 6D - Sustentabilidade:** Essa dimensão possibilita integrar informações relacionadas à proteção ambiental e ao consumo de energia, otimizando os custos e investimentos durante e após a construção.
- **BIM 7D - Gerenciamento de Instalações:** Um banco de dados detalhado é gerado para cada elemento da construção, incluindo estruturas, acabamentos e equipamentos. Assim, é possível programar manutenções, verificar garantias e gerenciar componentes ao longo da vida útil da edificação.

Devido às mudanças significativas que o BIM exige em comparação ao CAD, a resistência à sua implementação é compreensível. Conforme Coelho, o uso de ferramentas tradicionais, como o AutoCAD®, representa um obstáculo, pois a transição para novos *softwares* implica na reconfiguração de rotinas e no aprendizado de novas ferramentas (COELHO; NOVAES, 2018). Souza destaca ainda que a metodologia BIM exige *softwares* mais avançados, que geram volumes elevados de dados, demandando hardwares de alto desempenho, o que dificulta a adoção em muitas empresas (SOUZA, 2009).

Segundo Eastman, o conceito BIM não está vinculado a um único *software*, já que atender a todo o ciclo de vida de uma edificação em uma única aplicação seria inviável devido à complexidade e rigidez do processo (EASTMAN; TEICHOLZ; SACKS, 2011). Em vez disso, a criação de modelos BIM depende de diversas ferramentas especializadas, que frequentemente apresentam problemas de compatibilidade, dificultando a troca de dados (IBRAHIM ROBERT KRAWCZYK, 2004). Bottega relata que, mesmo após a adoção de plataformas BIM, a transferência de arquivos é prejudicada por questões como o tamanho dos dados e a diversidade de *softwares* utilizados (BOTTEGA, 2012).



Figura 1 – Representação do BIM - Fonte: HOCH

2.2 Aprendizado de Máquina

O Aprendizado de Máquina (AM) é um ramo da Inteligência Artificial (IA) que permite aos computadores aprenderem comportamentos, identificarem padrões e tomarem decisões com interação mínima de seres humanos (Mitchell, 1997). A partir de um conjunto de dados, os algoritmos passam por um processo de treinamento, gerando um modelo que possa lidar com novas situações e resolver problemas (TAN; STEINBACH; KUMAR, 2006).

Existem três principais tipos de aprendizagem:

- **Aprendizagem supervisionada:** ocorre quando um conjunto de dados previamente rotulados é utilizado para treinar o modelo, permitindo que ele preveja rótulos desconhecidos. Esse tipo de aprendizagem é capaz de tomar decisões precisas quando novos dados rotulados são fornecidos. Para estimar rótulos, isso pode ser feito por meio de dois tipos principais de tarefa: classificação e regressão. A classificação é utilizada para categorizar os dados em classes específicas, enquanto a regressão busca prever valores contínuos baseados em dados históricos (Faceli et al., 2011).
- **Aprendizagem não supervisionada:** ocorre quando um conjunto de dados não possui nenhum rótulo associado. Nesse contexto, o objetivo é identificar padrões ou similaridades entre os dados analisados. Essa forma de aprendizagem é amplamente aplicada em problemas de agrupamento, recomendações e detecção de anomalias. Nesse caso, mesmo que os dados não sejam rotulados, eles podem ser classificados como similares ou agrupados com base nos padrões identificados (Faceli et al., 2011).
- **Aprendizagem por reforço:** consiste em um processo de tentativa e erro, onde a máquina aprende quais ações resultam em maiores recompensas. A cada decisão tomada, o agente

recebe uma recompensa ou uma punição, e o objetivo é maximizar as recompensas ao longo do tempo. Essa abordagem é amplamente utilizada em jogos e aplicações onde a máquina precisa tomar decisões sequenciais, como, por exemplo, em sistemas de controle ou simulações em ambientes complexos (Cerri; Carvalho, 2017).

Nos problemas de classificação, o algoritmo busca criar um modelo capaz de generalizar os padrões extraídos do conjunto de treinamento para categorizar novas instâncias de dados não vistos. Já nos problemas de regressão, o objetivo do algoritmo é estimar valores contínuos, como, por exemplo, prever as vendas futuras de um produto ou estimar o preço de um imóvel com base em características históricas (Guimarães; Meireles; Almeida, 2019).

Um exemplo de problema de classificação é o mapeamento de uma imagem para classificar um indivíduo como homem ou mulher, ou a utilização de um classificador de sentimentos, onde o modelo pode determinar se um texto é positivo ou negativo. Por outro lado, exemplos de problemas de regressão incluem algoritmos que preveem vendas de produtos, estimam o valor de um imóvel ou calculam a expectativa de vida de um país.

Esse processo de aprendizagem é estruturado em etapas conhecidas como Fluxo de Aprendizagem de Máquina, que incluem: obtenção dos dados; preparação, limpeza e manipulação dos dados; escolha e treinamento do modelo; teste dos dados; avaliação do modelo; e melhorias do modelo (TAN; STEINBACH; KUMAR, 2006). A qualidade dos dados desempenha um papel fundamental no AM, pois o conhecimento extraído é diretamente influenciado pela qualidade das informações fornecidas (Guimarães; Meireles; Almeida, 2019). Portanto, é necessário um processo rigoroso de pré-processamento dos dados antes da aplicação dos algoritmos de AM.

O pré-processamento de dados é essencial, especialmente devido à presença de ruídos nos dados. Essa etapa pode incluir a detecção e remoção de anomalias, o tratamento de valores ausentes ou até mesmo a exclusão de atributos irrelevantes. Todo esse esforço visa aumentar a qualidade do treinamento do modelo. O sucesso do pré-processamento depende diretamente da habilidade do profissional em identificar problemas nos dados e utilizar métodos adequados para solucioná-los (Neves, 2003).

Além das dificuldades no pré-processamento, também podem surgir problemas nas fases de treinamento e teste. Um dos desafios mais comuns é o overfitting, que ocorre quando o modelo se ajusta excessivamente aos dados de treinamento, tornando-se incapaz de generalizar para novos dados. Outro problema é o underfitting, quando o modelo não consegue capturar padrões suficientes nos dados, resultando em um desempenho insatisfatório (RUSSELL; NORVIG, 2010).

Uma vez que o modelo é construído, torna-se essencial medir seu desempenho. Entre as métricas mais comuns utilizadas para esse propósito estão:

- **A acurácia (ACC ou *accuracy*):** É uma métrica muito utilizada na literatura, e sua fórmula calcula o número de acertos dividido pelo número total (número de acertos +

número de erros). Essas informações são retiradas da matriz de confusão (Equação 2.1). A matriz de confusão (Figura 2) é uma tabela que mostra o desempenho de um algoritmo de classificação. Se o problema de classificação for binário, ela apresentará quatro informações, que são:

$$ACC = \frac{(TP + TN)}{(TP + FN + FP + TN)} \quad (2.1)$$

- **Verdadeiro Positivo (TP ou *True Positive*)**: Ocorre quando a classe que estamos buscando foi prevista corretamente. Exemplo: a pessoa está doente, e o modelo previu corretamente que está doente;
- **Falso Positivo (FP ou *False Positive*)**: Ocorre quando a classe que estamos buscando foi prevista incorretamente. Exemplo: a pessoa não está doente, mas o modelo previu incorretamente que está doente;
- **Verdadeiro Negativo (TN ou *True Negative*)**: Ocorre quando a classe que não estamos buscando prever foi prevista corretamente. Exemplo: a pessoa não está doente, e o modelo previu corretamente que não está doente;
- **Falso Negativo (FN ou *False Negative*)**: Ocorre quando a classe que estamos buscando foi prevista incorretamente. Exemplo: a pessoa está doente, mas o modelo previu incorretamente que não está doente.

Figura 2 – Matriz de confusão.

		VALOR VERDADEIRO	
		positivo	negativo
VALORES PREVISTOS	positivo	TP	FN
	negativo	FP	TN

Fonte: Própria (2024).

- A **precisão (PRE ou *precision*)**: É utilizada para trabalhar somente com os valores positivos, e sua fórmula é calculada pelos valores verdadeiros positivos divididos pelo total de predições positivas (verdadeiro positivo + falso positivo) (Equação 2.2).

$$PRE = \frac{TP}{(TP + FP)} \quad (2.2)$$

- **A revocação (REC ou *recall*):** É utilizada para encontrar exemplos de uma classe, e sua fórmula é calculada pelos valores verdadeiros positivos divididos pelo número de resultados que deveriam ser apresentados (verdadeiro positivo + falso negativo) (Equação 2.3).

$$REC = \frac{TP}{(TP + FN)} \quad (2.3)$$

- **A medida F1 (F1S ou *F1-Score*):** É a junção da precisão com a revocação, resultando em um número único que indica a qualidade geral do modelo (Equação 2.4). O cálculo da medida F1 é dado por:

$$F1S = \frac{2 \cdot PRE \cdot REC}{PRE + REC} \quad (2.4)$$

Outro conceito relevante no contexto de AM é a transferência de aprendizado (Transfer Learning), que envolve a reutilização de modelos treinados para resolver problemas relacionados em outros domínios. Essa abordagem busca aproveitar o conhecimento adquirido em uma tarefa para facilitar o aprendizado em outra tarefa, mesmo que os conjuntos de dados sejam diferentes (Shao; Zhu; Li, 2015).

Finalmente, o Aprendizado Profundo (*Deep Learning*) é uma técnica avançada de AM que utiliza redes neurais profundas, inspiradas nos neurônios do cérebro humano. Essas redes são compostas por camadas de entrada, ocultas e saída, e transformam dados em informações úteis por meio de processos sequenciais. O aprendizado profundo é amplamente utilizado em aplicações como reconhecimento de imagens, processamento de linguagem natural e sistemas de recomendação (Arroyo-Figueroa; Sucar; Villavicencio, 2000)

2.3 PLN

Na década de 1950, Alan Turing (conhecido como o pai da computação) propôs o Teste de Turing, um experimento no qual um sistema tenta determinar se está interagindo com um ser humano ou uma máquina. Esse teste marcou o início das reflexões sobre o Processamento de Linguagem Natural (PLN) (RUSSELL; NORVIG, 2010).

O Processamento de Linguagem Natural (PLN) tem como objetivo permitir que computadores realizem tarefas relacionadas à linguagem humana, criando uma interação entre máquinas e humanos por meio de linguagem falada e/ou escrita (Jurafsky; Martin, 2009). Segundo Liddy, o PLN é uma técnica que analisa e representa textos de maneira natural, buscando alcançar um processamento de linguagem semelhante ao do ser humano em diversas aplicações (Liddy, 1998).

Entretanto, as linguagens naturais, como o Português e o Inglês, não possuem regras determinísticas e apresentam ambiguidades. Isso as diferencia das linguagens formais, como

Python ou Java, que possuem conjuntos de regras bem definidos. Nas linguagens formais, frases precisam seguir a gramática para serem válidas, e sua semântica é projetada para evitar ambiguidades.

Os sistemas de PLN podem ser classificados de acordo com o nível de unidade linguística processada ([Liddy, 1998](#)), como segue:

- **Nível Fonológico:** Interpretação dos sons da língua. Um exemplo é a diferenciação entre as palavras "faca" e "vaca" por meio do som.
- **Nível Morfológico:** Análise da função, estrutura e flexão das palavras. Por exemplo, identificar que "andar" é um verbo que indica ação.
- **Nível Lexical:** Análise do significado das palavras, incluindo suas relações semânticas. Por exemplo, compreender que "pedra" está associada a palavras como "pedregulho", "pedraria" e "pedrinha".
- **Nível Sintático:** Estudo da estrutura das frases e de como são organizadas para transmitir significado.
- **Nível Semântico:** Análise do significado das palavras e frases, como identificar que o verbo "levar" pode significar transportar, carregar ou guiar, dependendo do contexto.
- **Nível de Discurso:** Interpretação de textos mais extensos, considerando sua estrutura e significado.
- **Nível Pragmático:** Compreensão do contexto comunicacional. Por exemplo, quando alguém diz "Nossa, as janelas estão todas abertas! Como está frio aqui!" e outra pessoa responde "Vou fechá-las", é possível inferir a intenção implícita da primeira pessoa.

De acordo com Bulengo, o PLN pode ser dividido em quatro etapas principais: análise morfológica, análise sintática, análise semântica e análise pragmática, que são executadas nessa ordem ([Bulegon; Moro, 2010](#)). Santos, no entanto, afirma que geralmente são utilizadas apenas as três primeiras análises. Na análise morfológica, por exemplo, identificam-se substantivos, verbos e adjetivos, gerando um "dicionário" que é usado na análise sintática para compreender as relações entre palavras, como sujeito, predicado e complementos. Por fim, a análise semântica examina os termos ambíguos e seus significados no contexto ([Santos et al., 2015](#)).

Na prática, raramente todos os níveis são implementados em um sistema, devido à complexidade crescente à medida que se avança nos níveis. No entanto, o PLN tem ampla aplicação, como em reconhecedores e sintetizadores de fala, corretores ortográficos, tradutores automáticos, sistemas de geração e resumo de texto, extração de informações e interfaces de linguagem natural ([Vieira; Lima, 2001](#)).

Com o crescimento do PLN, pesquisadores têm buscado fundamentação em diversas áreas, como Filosofia da Linguagem, Psicologia, Matemática, Linguística e Ciência da Computação. Assim como no Aprendizado de Máquina (AM), o PLN exige etapas de pré-processamento que estruturam a linguagem, reduzindo o vocabulário e melhorando o desempenho dos modelos (Rezende; Marcacini; Moura, 2011).

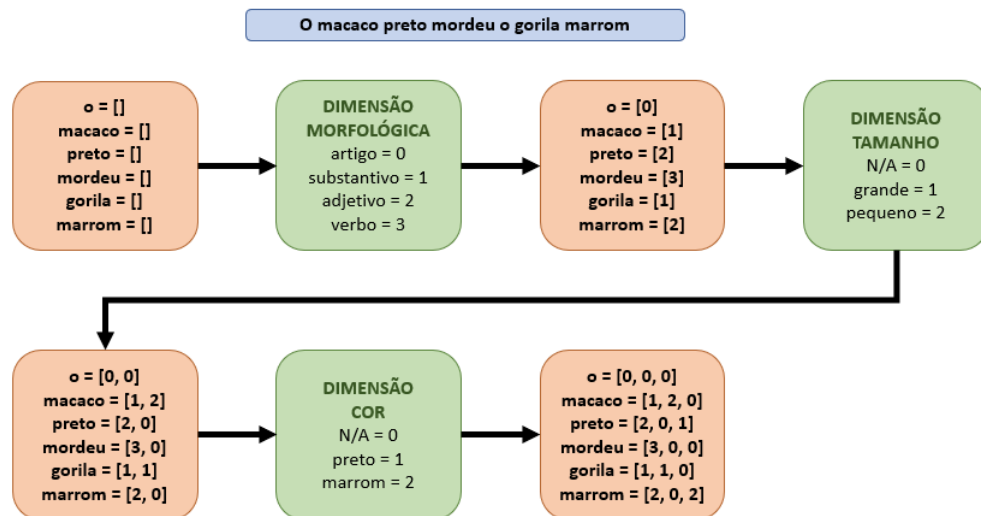
Entre as principais tarefas de pré-processamento no PLN, destacam-se:

- **Normalização:** Remoção de caracteres especiais, tags HTML/CSS, e transformação de letras maiúsculas em minúsculas.
- **Tokenização:** Separação do texto em unidades mínimas chamadas "tokens", que podem ser palavras, símbolos ou caracteres.
- **Stemização:** Redução das palavras ao seu radical, como transformar "meninas" em "menin".
- **Lematização:** Redução das palavras à sua forma base ou lema, como transformar "meninas" e "meninos" em "menino".
- **Remoção de Stopwords:** Exclusão de palavras frequentes e pouco informativas, como "a", "de", "e".
- **Correção Ortográfica:** Tratamento de erros de digitação e abreviações para evitar problemas nos modelos.

A representação por palavras é conhecida como vetores de palavras (ou *word embeddings*), onde a ideia é transformar as palavras em números, mantendo suas relações com outras palavras. Esse processo se baseia em um *corpus* (um conjunto de textos a partir do qual as relações serão aprendidas). A primeira etapa é dividir o texto em palavras; em seguida, cada uma delas é representada por vetores, onde o tamanho do vetor corresponde ao número de dimensões definidas pelo *corpus*. A Figura 3 ilustra o *word embeddings* com 3 dimensões (morfológica, tamanho e cor), representando um conceito completo.

As vantagens do *word embeddings* incluem:

- **Representação Sem Contexto:** As palavras são analisadas independentemente do contexto imediato, o que pode limitar certas nuances, mas ainda permite categorizações úteis.
- **Relações Semânticas:** Em um espaço n-dimensional, palavras com significados similares estão próximas umas das outras, como "macaco" e "gorila".
- **Operações Matemáticas:** É possível realizar operações vetoriais para descobrir relações semânticas entre palavras. Por exemplo, somar um vetor associado a uma característica específica a outro vetor pode resultar em uma nova palavra relacionada.

Figura 3 – Exemplo da transformação de palavras em *word embeddings*.

Fonte: Própria (2021).

Enquanto no exemplo acima a intuição foi usada para criar as dimensões, na Inteligência Artificial modelos baseados em Aprendizado Profundo são comumente aplicados. Eles classificam palavras em dimensões com base no contexto em que aparecem. Quanto maior o *corpus*, mais precisas e eficientes tendem a ser essas representações (Bengio et al., 2003).

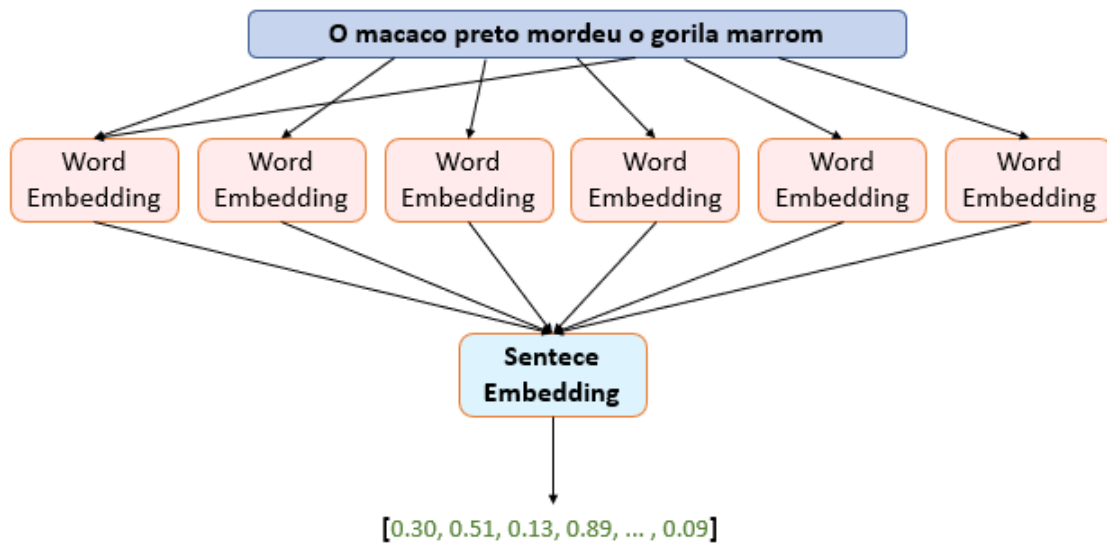
Além do conceito de *word embeddings*, existe o *sentence embeddings*, que considera não apenas as palavras, mas também a ordem em que aparecem na frase (Conneau et al., 2017). No *sentence embeddings*, cada palavra é representada por seu *word embedding*, e o conjunto dessas palavras forma uma frase completa, com informações semânticas representadas por vetores, facilitando a compreensão do contexto e da intenção da frase (Figura 4).

Diversas técnicas foram desenvolvidas para representar palavras vetorialmente (*word embeddings*), diferenciando-se por suas arquiteturas e bases de dados usadas no treinamento (Martin; Jurafsky, 2009). Além disso, essas técnicas podem ser transferidas para domínios semelhantes em um processo chamado Transferência de Aprendizado, que será discutido em outra seção.

No contexto do PLN moderno, representações vetoriais, como *word embeddings* e *sentence embeddings*, têm sido amplamente utilizadas. Modelos como Word2Vec, GloVe, fastText e OpenAIEmbeddings destacam-se no campo (Devlin et al., 2019). O OpenAIEmbeddings, em particular, apresenta um desempenho robusto em diversas tarefas de PLN.

2.4 Redes Neurais Artificiais

O sistema nervoso humano é composto por neurônios, que desempenham a função de transmitir informações essenciais para o funcionamento, comportamento e raciocínio do

Figura 4 – Exemplo da transformação de *word embeddings* para *sentence embeddings*.

Fonte: Própria (2021).

corpo humano (Tafner; Xerez; Filho, 1995). As Redes Neurais Artificiais (RNAs) são técnicas computacionais inspiradas na estrutura neural do ser humano, adquirindo conhecimento por meio da experiência (Haykin, 1999). Assim, uma rede neural artificial pode ser compreendida como uma abstração da rede neural biológica, com o objetivo de modelar processos de aprendizado e resolver problemas complexos.

O primeiro modelo que simulou uma rede neural biológica foi desenvolvido em 1943 pelos psiquiatras Warren McCulloch e Walter Pitts. Esse trabalho apresentou detalhes sobre o comportamento e a memória dos neurônios biológicos em uma forma artificial, mas não abordou como o aprendizado poderia ser implementado (McCulloch; Pitts, 1943)). Já em 1949, Donald Hebb conduziu o primeiro estudo voltado à fase de aprendizado das redes neurais artificiais, demonstrando que os pesos em cada conexão entre neurônios resultavam em uma força de excitação, similar à encontrada nos neurônios biológicos (Hebb, 1949). Entretanto, esse modelo ainda não possuía a capacidade de reconhecer padrões.

Em 1958, Rosenblatt desenvolveu o primeiro modelo capaz de detectar padrões e características, demonstrando que, ao modificar os pesos conectados aos neurônios, o modelo poderia ser treinado para classificar padrões (Rosenblatt, 1958). Este modelo ficou conhecido como perceptron e apresenta uma arquitetura composta por três camadas:

- **Camada de entrada:** Responsável por receber informações do ambiente externo e repassá-las às camadas intermediárias.
- **Camadas intermediárias (ou ocultas):** Processam os sinais recebidos, ajustam os pesos e enviam as informações para a camada seguinte.

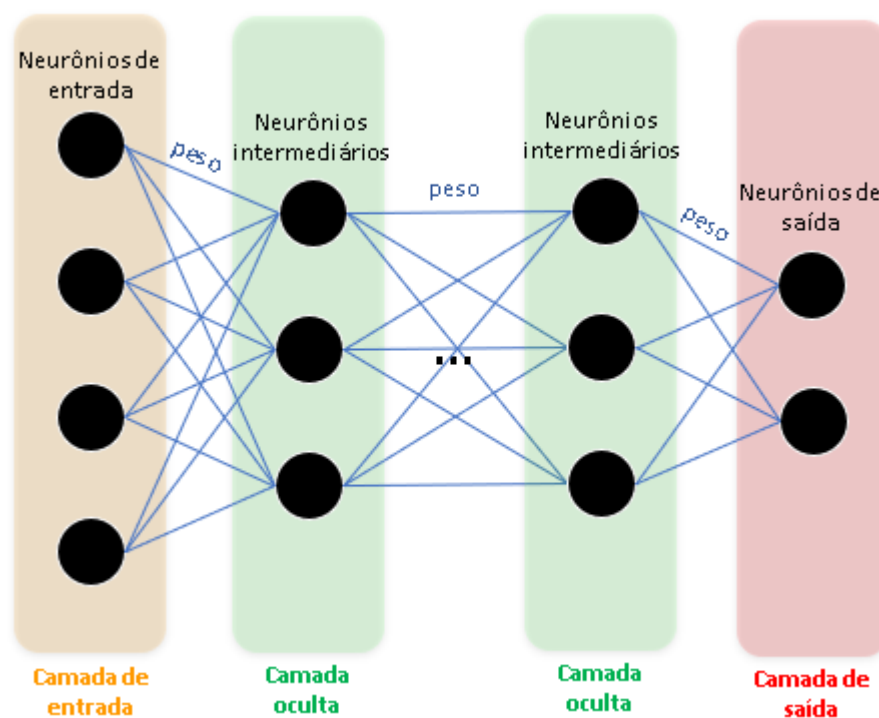
- **Camada de saída:** Produz e apresenta o resultado final.

Contudo, quando utilizado para classificar padrões, uma rede perceptron de três camadas é limitada a encontrar uma única solução. Para resolver problemas que exigem múltiplas soluções, Minsky e Papert propuseram a inclusão de múltiplas camadas intermediárias, resultando no modelo conhecido como perceptron multicamadas (*Multi-layer Perceptron* – MLP) (Minsky; Papert, 1969).

O MLP é composto por um conjunto de neurônios conectados em paralelo, com cada conexão associada a um peso que reflete sua influência na rede. A partir da interação em camadas intermediárias, onde a saída de um neurônio se torna a entrada de outro, obtém-se um comportamento inteligente, capaz de reconhecer padrões e características (Soares; Teive, 2015).

A estrutura de um neurônio em uma rede MLP processa informações recebidas do ambiente externo (valores x) associadas a pesos (w). Uma soma ponderada dos produtos de x e w é então calculada, somando-se o limiar de ativação (bias ou polarização). O valor resultante, chamado de ponto de ativação (u), é avaliado por uma função de ativação que limita a amplitude da saída do neurônio, geralmente dentro do intervalo $[0, 1]$, podendo ser interpretado como uma probabilidade (Goodfellow; Bengio; Courville, 2016).

Figura 5 – Rede Neural MLP.

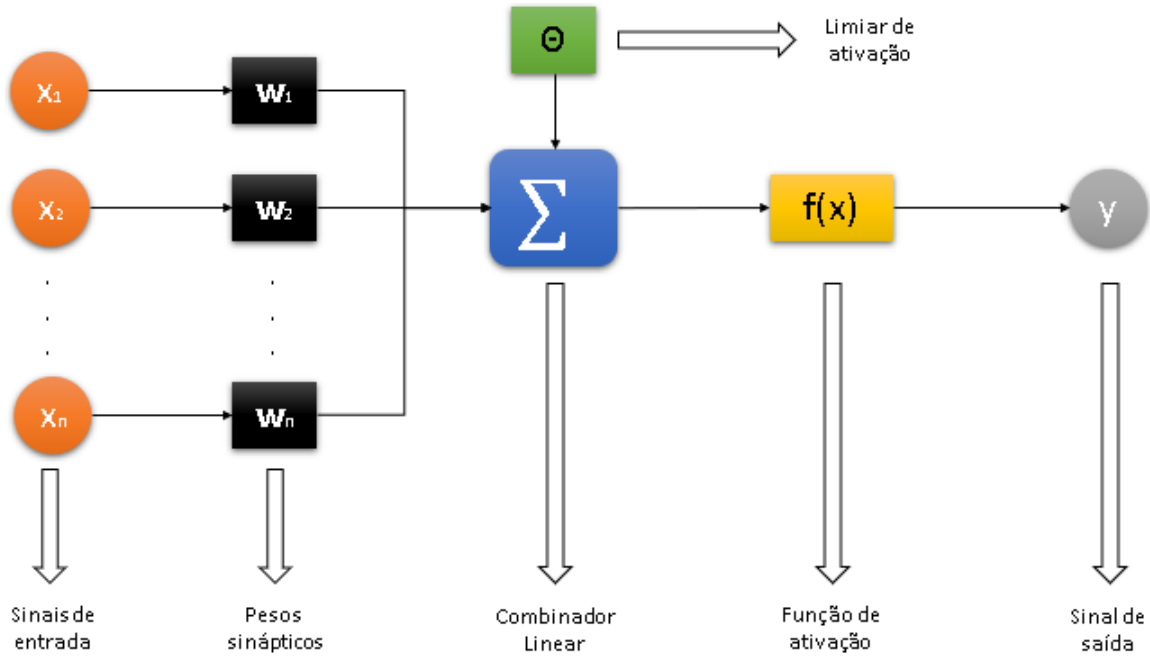


Fonte: Própria (2024).

A Figura 6 demonstra o processamento de informações por um neurônio em uma rede MLP, onde, dado um conjunto de valores ' x ' (que representam as informações do mundo externo),

esses valores são associados a pesos ' w '. A partir disso, é efetuada uma soma ponderada dos produtos ' x ' e ' w ', à qual é somado o limiar de ativação (bias ou polarização), como mostrado na Equação 2.5.

Figura 6 – Estrutura de um neurônio no MLP.

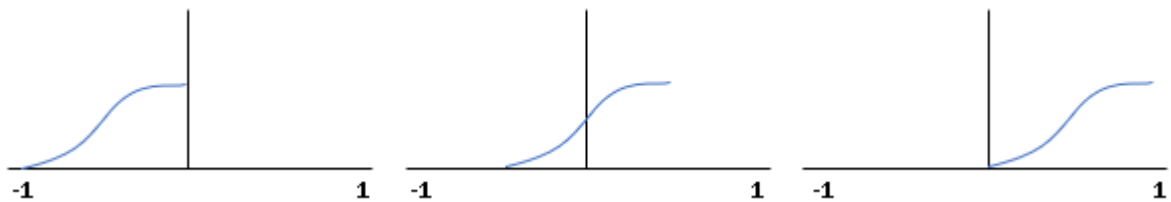


Fonte: Própria (2024).

$$u = \sum_{i=1}^n x_i \cdot w_i + \theta \quad (2.5)$$

O bias (θ) é responsável pelo deslocamento inicial da função de ativação no plano onde ela é representada. Caso o bias seja positivo, o movimento é realizado para a esquerda, diminuindo o valor do eixo x. Caso seja negativo, o movimento é realizado para a direita, aumentando o valor do eixo x (Rojas, 1996), como ilustrado na Figura 7.

Figura 7 – Exemplo do deslocamento do gráfico através da mudança da constante bias.



Fonte: Própria (2024).

O valor u obtido, chamado de ponto de ativação, é avaliado pela função de ativação, que é responsável por calcular o sinal da saída do neurônio. A função de ativação, também conhecida

como função de transferência, tem o objetivo de limitar a amplitude da saída do neurônio. Ou seja, o valor obtido estará no intervalo entre 0 e 1, podendo ser interpretado como uma probabilidade do resultado (Goodfellow; Bengio; Courville, 2016).

Existem diversos tipos de funções de ativação, e não há uma regra fixa para determinar qual é melhor ou pior. A escolha depende do problema, embora, dependendo das propriedades específicas, seja possível optar por uma função que facilite e acelere a convergência da rede neural (Goodfellow; Bengio; Courville, 2016).

As funções de ativação mais populares são:

- **Função de Etapa Binária** ($f(x) = 1$, se $x \geq 0$ e $f(x) = 0$, se $x < 0$): Essa função determina se o neurônio deve ou não estar ativo. Caso o valor 'u' seja maior que um limite pré-determinado, o neurônio é ativado; caso contrário, ele permanece desativado. Embora útil em classificadores binários, é mais utilizada como referência teórica, pois, no processo de retropropagação (backpropagation), os gradientes das funções de ativação retornam zero, o que impede a melhoria dos resultados (Feng; Lu, 2019).
- **Função Linear** ($f(x) = ax$): Nessa função, cada camada passa por uma transformação linear. No entanto, essa função possui limitações semelhantes à função de etapa binária, já que sua derivada é constante, independentemente do valor da entrada. Isso impede melhorias durante o backpropagation (Feng; Lu, 2019).
- **Função Sigmoid** ($f(x) = 1/(1 + e^{-x})$): Essa função comprime os valores de saída para o intervalo entre 0 e 1, o que é útil para classificar uma classe específica. Embora não seja linear e permita saídas não lineares em redes neurais, apresenta dois problemas principais: 1) os gradientes podem se aproximar de zero, o que prejudica o aprendizado da rede, e 2) os valores de saída são sempre positivos, o que pode não ser desejável em todos os casos. Apesar disso, ela é amplamente utilizada em classificadores (Feng; Lu, 2019).
- **Função Tanh** ($f(x) = 2/(1 + e^{-2x}) - 1$): É uma variação escalonada da função Sigmoid, mas simétrica em relação à origem, variando de -1 a 1. Isso elimina o problema dos valores positivos constantes. As demais propriedades são semelhantes às da Sigmoid (Feng; Lu, 2019).
- **Função ReLU** ($f(x) = \max(0, x)$): É a função de ativação mais amplamente utilizada no design de redes neurais. Por ser não linear, facilita a propagação dos erros e melhorias durante o treinamento. Sua principal vantagem é não ativar todos os neurônios simultaneamente, tornando a rede mais eficiente. No entanto, pode sofrer com o problema de gradientes que convergem para zero (Feng; Lu, 2019).
- **Função Leaky ReLU** ($f(x) = ax$, se $x < 0$ e $f(x) = x$, se $x \geq 0$): Uma variação da ReLU que corrige o problema do gradiente zero. Quando 'x' < 0, é aplicada uma componente

linear em vez de zero, garantindo um gradiente constante mesmo para valores negativos. É especialmente útil em camadas ocultas e para lidar com neurônios defeituosos (Feng; Lu, 2019).

- **Função Softmax:** É uma extensão da Sigmoid, projetada para problemas de classificação com múltiplas classes. Transforma as saídas de cada classe em valores entre 0 e 1, normalizando-as para que sua soma seja igual a 1. Assim, é possível interpretar os resultados como probabilidades de pertencimento a cada classe (Sensoy; Kaplan; Kandemir, 2018).

O treinamento de uma rede neural artificial multicamadas consiste em ajustar os pesos sinápticos e o bias de modo que as saídas se aproximem das saídas esperadas. Esse processo ocorre em duas etapas (RUSSELL; NORVIG, 2010):

- **Propagação (forward-propagation):** Nesta etapa, aplica-se a Equação 3.7 em cada neurônio. O fluxo de dados começa na camada de entrada, atravessa as camadas ocultas e finaliza na camada de saída (Hafemann, 2014).
- **Retropropagação (backpropagation):** Após a etapa de propagação, os valores de saída são comparados com os valores desejados usando uma função custo, que mede o desempenho da rede. Caso o desempenho seja insatisfatório, inicia-se a retropropagação para minimizar o erro, ajustando os pesos e os bias. Isso é feito utilizando o método do Gradiente Descendente (Hafemann, 2014).
 - **Função Custo (Loss):** Mede o quão distante está a saída da rede em relação à saída esperada. Para regressão, a função mais comum é o Erro Quadrático Médio (MSE). Para classificação, é a Entropia Cruzada (McCaffrey, 2013).
 - **Erro Quadrático Médio (MSE):** Mede a diferença entre os valores previstos e os reais, elevando ao quadrado e calculando a média (Equação 2.6) (Willmott, 1981).

$$MSE = \frac{1}{n} \sum_{i=1}^n (\theta_{ip} - \theta_i)^2 \quad (2.6)$$

- **Raiz do Erro Quadrático Médio (RMSE):** Similar ao MSE, mas calcula a raiz quadrada da média dos erros (Equação 2.7) (Chai; Draxler, 2014).

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n e_i^2} \quad (2.7)$$

- **Erro Médio Absoluto (MAE):** Mede o desvio médio entre valores previstos e reais, com pesos iguais para todos os desvios (Equação 2.8) (Willmott, 1981).

$$MAE = \frac{\sum_{i=1}^n |O_i - P_i|}{n} \quad (2.8)$$

- **Entropia Cruzada:** Mede a distância entre os vetores de saída e os valores esperados (Equação 2.9) (Nasr; Badr; Joun, 2002).

$$E(w) = \frac{1}{n} \cdot \sum_{i=1}^n [t_i \cdot \log(y_i) + (1 - t_i) \cdot \log(1 - y_i)] \quad (2.9)$$

- **Gradiente Descendente:** Método de otimização que ajusta pesos e bias na direção que minimiza o erro. A cada iteração, busca-se o mínimo local da função custo (Equação 2.10) (Rumelhart; Hinton; Williams, 1986).

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \partial_{(\mathbf{w}, b)} l^{(i)}(\mathbf{w}, b) \quad (2.10)$$

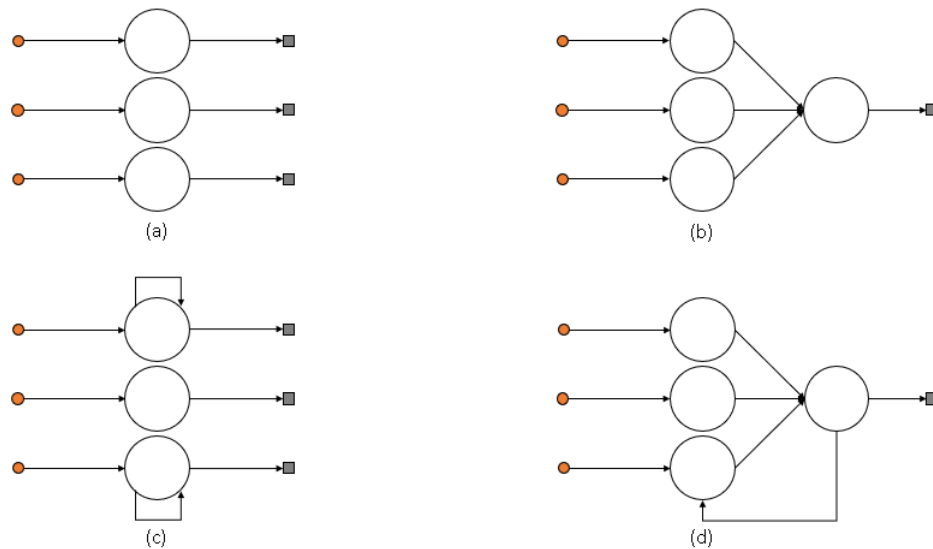
Cada ciclo de propagação e retropropagação ajusta os pesos para reduzir a função de perda, repetindo o processo até atingir o erro mínimo local (RUSSELL; NORVIG, 2010).

Um dos ramos do Aprendizado de Máquina baseado em Redes Neurais Artificiais é o Aprendizado Profundo (ou Deep Learning), que utiliza diversas camadas capazes de extrair características em diferentes níveis. Por exemplo, em um processamento de imagem, a primeira camada pode extrair formas gerais, como objetos ou rostos humanos, enquanto camadas posteriores extraem características mais específicas, como bordas e cantos. Ou seja, as camadas iniciais extraem recursos de baixo nível, enquanto as camadas mais profundas extraem recursos de alto nível e/ou médio nível (Schmidhuber, 2015).

Um dos principais benefícios da Aprendizagem Profunda é sua capacidade de extrair características automaticamente, utilizando pesos adaptáveis no kernel, o que economiza tempo ao evitar a necessidade de pré-processamento manual (Schmidhuber, 2015). Nesse contexto, uma rede neural artificial possui três tipos de camadas, e os nós (ou nodos) podem ter dois tipos de conexões, que são:

- **Redes de camada única:** ocorre quando há apenas um nó entre qualquer entrada e saída da rede (Figura 8 (a) e (c)) (Braga; Ludermir; Carvalho, 2000).
- **Redes de múltiplas camadas:** ocorre quando há mais de um neurônio entre qualquer entrada e saída da rede (Figura 8 (b) e (d)) (Braga; Ludermir; Carvalho, 2000).
- **Redes recorrentes:** ocorre quando existe pelo menos um laço de realimentação em alguma camada (Figura 8 (c) e (d)) (Braga; Ludermir; Carvalho, 2000).
- **Nó acíclico (feedforward):** ocorre quando a saída de um neurônio não é utilizada como entrada em camadas anteriores ou iguais (Figura 8 (a) e (b)) (Haykin, 2001).
- **Nó cíclico (feedback):** ocorre quando a saída de um neurônio é utilizada como entrada em camadas anteriores ou iguais (Figura 8 (c) e (d)) (Haykin, 2001).

Figura 8 – Tipos de camadas e conexões de nodos.



Fonte: Própria (2024).

Ou seja, Redes Neurais Recorrentes (Recurrent Neural Networks ou RNN) diferem substancialmente das tradicionais Redes Neurais Feedforward (FNN), pois possuem ciclos, permitindo que os dados de saída sejam alimentados de volta nas camadas anteriores. Isso torna o modelo capaz de capturar dependências temporais entre os dados (Ramakrishnan; Soni, 2018).

O ciclo presente nas RNNs permite que essas redes utilizem seu estado interno (memória) para processar entradas sequenciais, o que é essencial para tarefas que envolvem sequências temporais. Essa capacidade de armazenar informações passadas é especialmente útil em aplicações como reconhecimento de fala, processamento de linguagem natural, previsão de séries temporais, entre outras.

No entanto, devido à repetição de operações no ciclo das RNNs, que inclui uma matriz de pesos W representando os pesos de cada neurônio em uma camada, essas redes tendem a se tornar profundas. Essa profundidade pode gerar problemas como a dissipação do gradiente (gradient vanishing) e a explosão do gradiente (gradient explosion) (Goodfellow; Bengio; Courville, 2016).

A dissipação do gradiente ocorre quando os erros das últimas camadas afetam pouco os erros nas camadas iniciais, prejudicando o treinamento da rede. Isso acontece porque, durante as atualizações, os pesos e o bias das camadas iniciais não são significativamente ajustados. Por outro lado, a explosão do gradiente faz com que os gradientes nas camadas iniciais fiquem muito grandes, tornando a rede instável (Goodfellow; Bengio; Courville, 2016).

Para mitigar esses problemas, os cientistas Hochreiter e Schmidhuber propuseram o LSTM (*Long Short-Term Memory* ou Memória de Longo e Curto Prazo). Nesse modelo, os neurônios tradicionais foram substituídos por células de memória, permitindo que as redes RNN superem as limitações associadas à dissipação e à explosão do gradiente (Hochreiter;

Schmidhuber, 1997).

2.5 Memória Longa de Curto Prazo

O LSTM é uma evolução do RNN. O funcionamento dessa rede é estruturado de forma em cadeia, onde cada camada representa um intervalo de tempo t e a rede possui células de memória, nas quais cada célula contém portões de entrada, de esquecimento e de saída.

- **Portão de entrada (*input gate*):** Tem o objetivo de atualizar, a partir do fluxo de dados de entrada, o estado de memória da célula (Abdel-Nasser; Mahmoud, 2019).
- **Portão de esquecimento (*forget gate*):** Tem o objetivo de controlar qual informação deve permanecer na memória (Abdel-Nasser; Mahmoud, 2019).
- **Portão de saída (*output gate*):** Tem o objetivo de determinar como a saída é influenciada pelo valor de entrada e pela memória armazenada (Abdel-Nasser; Mahmoud, 2019).

Segundo (Tai; Socher; Manning, 2015), a primeira coisa que a célula do LSTM faz é decidir qual informação será removida ou mantida no estado atual da célula. Essa operação é realizada pelo portão do esquecimento, que retorna um valor entre 0 (esquecer) e 1 (manter). Todo esse processo é calculado pela função *sigmoid* (Equação 2.11).

$$f_t = \text{sigmoid}(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2.11)$$

Em seguida, decide-se qual nova informação será armazenada. Essa decisão ocorre em duas etapas: a primeira determina qual valor deverá ser atualizado (calculada pela função *sigmoid*) (Equação 2.12), e a segunda cria um vetor de novos valores candidatos que podem ser adicionados ao estado (calculada pela função *tanh*) (Equação 2.13).

$$i_t = \text{sigmoid}(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2.12)$$

$$\tilde{C}_t = \text{tanh}(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (2.13)$$

Depois disso, multiplica-se o estado antigo pelos candidatos passados (C_{t-1}), para esquecer as informações desnecessárias, e multiplica-se o estado atualizado pelos candidatos atuais ($\tilde{C}_t$), para manter as informações úteis (Equação 2.14).

$$C_t = f_t \times C_{t-1} + i_t \times \tilde{C}_t \quad (2.14)$$

Por fim, a camada *sigmoid* decide quais informações da célula irão para a saída (Equação 2.15), e essas informações são multiplicadas pela função *tanh* aplicada aos candidatos atuais, permitindo que somente as informações relevantes sejam emitidas (Equação 2.16).

$$o_t = \text{sigmoid}(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (2.15)$$

$$h_t = o_t \times \tanh(C_t) \quad (2.16)$$

A LSTM possui uma estrutura computacional complexa que opera com base na ativação de diferentes funções, decidindo quais informações devem ser mantidas, alteradas ou descartadas. Essa rede tem alcançado grande sucesso na solução de tarefas sequenciais (Tai; Socher; Manning, 2015). Com esse sucesso, surgiu uma variante denominada *Bidirectional Long Short-Term Memory* (BiLSTM ou Memória Longa de Curto Prazo Bidirecional), uma rede composta por duas LSTMs funcionando em paralelo. Uma das redes percorre a sequência para frente (*Forward LSTM*), enquanto a outra percorre a sequência para trás (*Backward LSTM*), permitindo capturar informações passadas e futuras, o que aumenta o poder de aprendizado da rede (Graves; Jaitly; Mohamed, 2013).

Apesar de as redes neurais recorrentes (RNNs) apresentarem iterações eficientes, as redes *Long Short-Term Memory* (LSTM), projetadas para tarefas do tipo *sequence-to-sequence* (*seq2seq*), recebem uma sequência de dados e produzem outra. Nesse contexto, uma estrutura neural que tem ganhado destaque no mundo do Processamento de Linguagem Natural (PLN) é o *transformer*, devido ao seu desempenho em tarefas como traduções, sumarizações e legendamento de imagens, entre outras.

2.6 Transformers

Redes neurais recorrentes (Ramakrishnan; Soni, 2018), memória de curto e longo prazo (Hochreiter; Schmidhuber, 1997) e redes neurais recorrentes com portas (Chung et al., 2014), em particular, foram firmemente estabelecidas como abordagens de última geração em modelagem de sequência. Redes de modelagem de sequência (*seq2seq*) possuem uma abordagem na qual se utilizam modelos de Aprendizado de Máquina ou de Aprendizado Profundo para obter uma sequência de dados como entrada (em um domínio específico) e converte essa sequência para uma representação em outro domínio (Sutskever; Vinyals; Le, 2014).

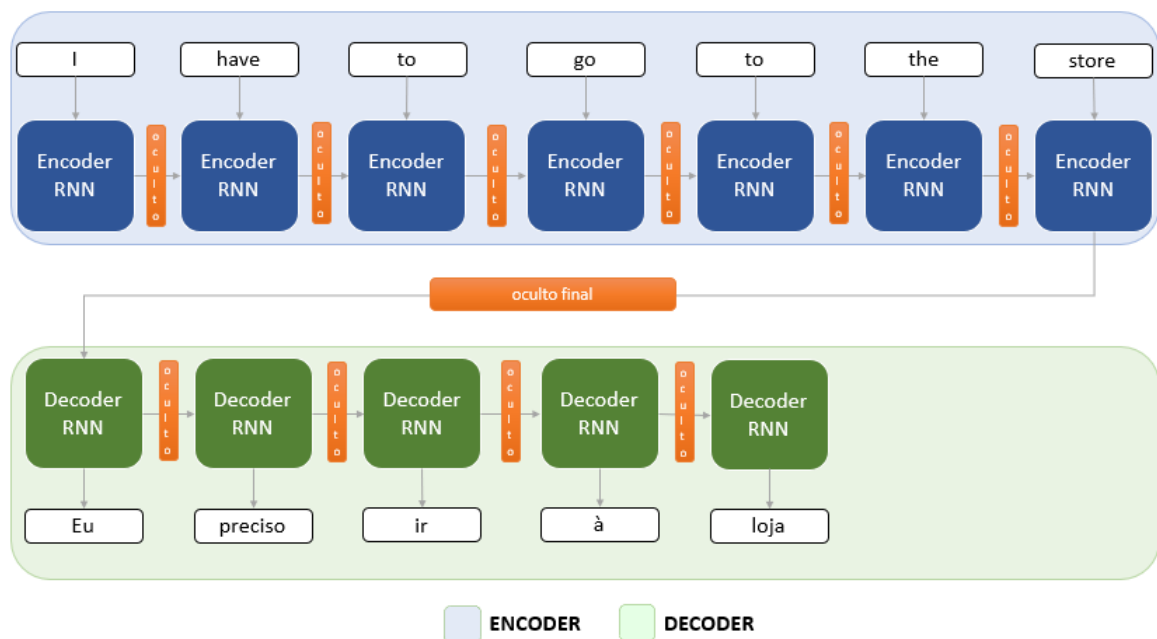
Essa abordagem normalmente é realizada por meio de dois módulos: um *encoder* (codificador) e um *decoder* (decodificador), onde o *encoder* tem o objetivo de processar a sequência de entrada e transformá-la em um contexto, enquanto o *decoder* pega o contexto e produz uma sequência de saída (Sutskever; Vinyals; Le, 2014). Um exemplo de *seq2seq* pode ser um modelo de tradução inglês-português, no qual o módulo *encoder* transforma o texto em

inglês em um "contexto livre de idioma" e, depois, o módulo *decoder* adiciona a segunda língua (português) nesse contexto.

No exemplo de um modelo de tradução inglês-português, será demonstrado o *encoder* e o *decoder* na forma de RNNs (Redes Neurais Recorrentes), onde o *encoder* processa as palavras da sequência de entrada uma a uma (de forma recorrente). A cada etapa do processamento, a palavra que entra na rede produz uma saída e um estado oculto (contexto). A saída geralmente do *encoder* é descartada, enquanto o estado oculto é passado para a próxima etapa, sendo utilizado nas palavras seguintes da sequência para produzir mais uma saída e criando um novo estado oculto (ou seja, atualizando o contexto). Esse processo se repete até a última palavra da sequência de entrada, onde o *encoder* cria o estado oculto final (contexto final).

Esse contexto final é passado para a segunda RNN, o *decoder*, que também recebe um "vetor genérico" (os valores são ajustados durante o treinamento). Esse vetor representa o início da frase de saída e, da mesma forma que o *encoder*, o *decoder* utiliza o vetor e o contexto para produzir uma saída e um estado atualizado, passando por etapas (nesse exemplo, palavras), e, em cada etapa, a saída representa a palavra traduzida até chegar na última etapa (palavra), que irá entregar a frase inteira traduzida (Figura 9).

Figura 9 – Exemplo de um modelo seq2seq para tradução do inglês para português.



Fonte: Própria (2024).

Porém, não é necessário utilizar apenas RNN para codificadores e decodificadores. É possível misturar e combinar diversos modelos, como demonstra o autor Jonghwan Mun, que utiliza um codificador CNN (*Convolutional Neural Networks*) (Simard; Steinkraus; Platt, 2003), um codificador RNN e um decodificador LSTM para um modelo que gera legendas de imagens (Mun; Cho; Han, 2016).

Como pode ser visto na Figura 9, a frase original tem 7 palavras, enquanto a frase traduzida contém 5 palavras. Por isso, dependendo da complexidade do problema, e no caso de traduções, a tarefa de traduzir não se trata de um simples alinhamento das palavras individuais. É necessário que o modelo seja capaz de focar em uma palavra, caso seja necessário, ou em um trecho específico da frase, criando assim o conceito de atenção.

Nas RNNs, o mecanismo de atenção geralmente é implementado da seguinte forma: no *encoder*, além do contexto ser representado pelo estado oculto final, ele também é representado por todos os estados ocultos produzidos após o processamento de cada palavra, fazendo com que cada estado oculto contenha o contexto acumulado até aquela palavra, o qual, após processar todas as palavras, é enviado para o *decoder* juntamente com todos os estados ocultos (Bahdanau; Cho; Bengio, 2015). Ou seja, no exemplo da Figura 9, a palavra 'go' após ser processada, possuirá o estado oculto (contexto) acumulado das palavras 'I', 'have', 'to' e 'go'.

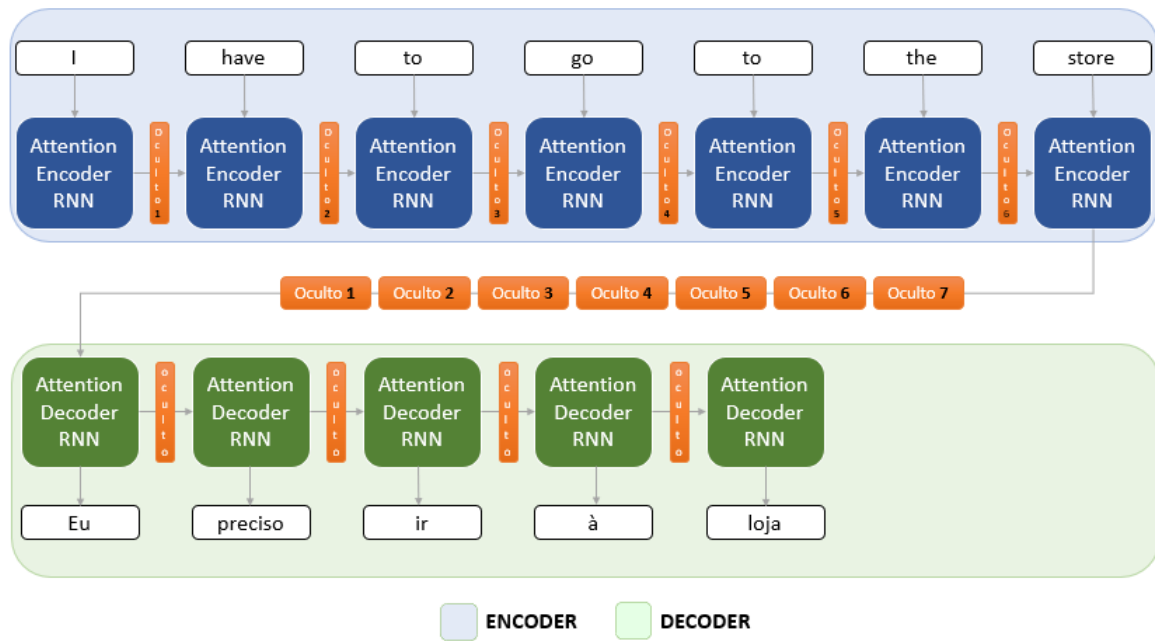
O *decoder* recebe e extrai do estado oculto final as informações necessárias para produzir o próximo estado oculto e utiliza o acúmulo de estados ocultos anteriores para calcular uma pontuação de atenção, indicando quanto foco a etapa dará para esses contextos parciais. O resultado da pontuação de atenção é multiplicado por cada estado oculto respectivo, e depois todos são somados, gerando o resultado denominado estado de contexto (Figura 10). Esse estado de contexto se espera que carregue a informação de interesse de forma amplificada, ou seja, indicando qual parte da frase o modelo deve prestar mais atenção. Por fim, o estado de contexto é concatenado com o estado final produzido nesta última etapa do *decoder*, onde o vetor é processado por uma estrutura de rede adicional do tipo totalmente conectada, produzindo então o vetor que representa a palavra no segundo idioma (português) (Bahdanau; Cho; Bengio, 2015).

De acordo com (Bahdanau; Cho; Bengio, 2015), o mecanismo de atenção obtém melhores resultados para a maioria das tarefas que necessitam "traduzir" uma sequência em outra, podendo transformar um texto em seu resumo, ou uma imagem em sua legenda. Porém, modelos recorrentes, mesmo utilizando o mecanismo de atenção (*Attention*), dada a sua natureza sequencial, acabam realizando processos computacionais intensivos para captar o contexto de uma sequência. Por isso, foi criada a arquitetura *transformer*, uma técnica que proporciona melhorias significativas tanto na eficiência computacional quanto na performance dos modelos baseados em arquiteturas recorrentes.

Assim como outras redes com mecanismo de atenção, o *transformer* também possui os módulos *encoder* e *decoder*, onde os criadores do modelo criaram 6 pilhas (*stacks*) de *encoders* e *decoders* individuais, arrançados linearmente. Cada *encoder* de cada pilha possui duas unidades (camadas) chamadas de *self-attention*, uma aplicando o mecanismo de atenção sobre o texto que ela recebe e outra que transforma o resultado da camada de *self-attention* em um *embedding* de comprimento menor (Vaswani et al., 2017).

O *decoder* é construído com as mesmas unidades do *encoder*, porém, possui uma camada adicional denominada *encoder-decoder attention*, cujo objetivo é mapear o resultado da camada

Figura 10 – Exemplo de um modelo com o mecanismo de atenção para tradução do inglês para português.

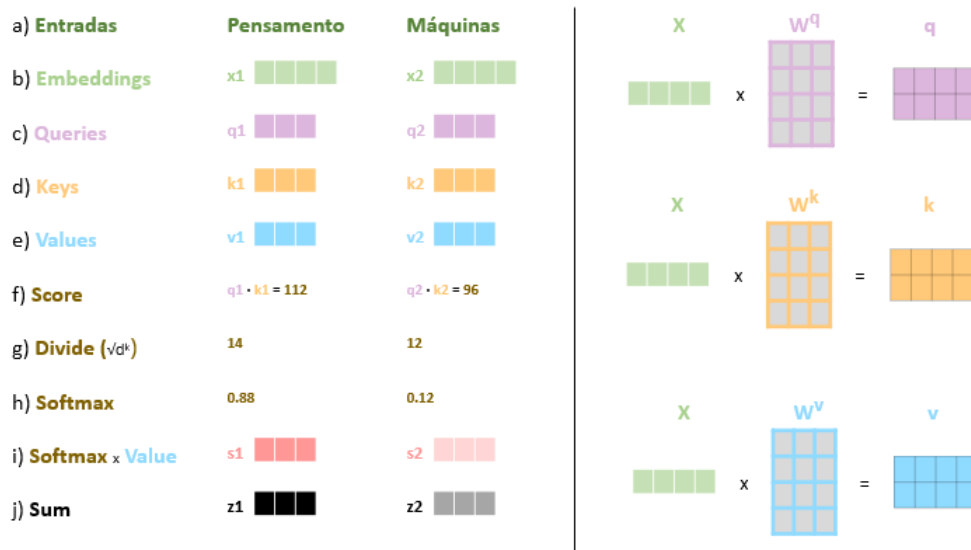


Fonte: Própria (2024).

de *self-attention* do *decoder* com os *embeddings* construídos pelo *encoder* (Vaswani et al., 2017). Com isso, enquanto nos modelos *seq2seq* as palavras são processadas sequencialmente, o *transformer* recebe o texto que deve processar tudo de uma vez, onde, através de um método independente, todas as palavras são convertidas em *embeddings*.

A camada *self-attention* (Figura 11) converte cada palavra em três vetores, denominados q (*query*), k (*key*) e v (*value*), onde é feita a multiplicação dos vetores da palavra por três matrizes (pesos) W^q , Q^k e W^v , respectivamente (Figura 11 (b), (c) e (d)), cujos parâmetros são ajustados durante o treinamento da rede. Esses vetores (q , k e v) são abstrações para permitir o cálculo de atenção, que é realizado por meio de uma multiplicação entre as *queries* e as *keys*. Essa operação irá resultar em uma pontuação que será normalizada e aplicada às *values*, como mostrado na Figura 11 (e). O resultado final de uma camada de *self-attention* é uma nova sequência que leva em consideração o foco nas palavras mais importantes de toda a sequência.

Essa pontuação é dividida pela raiz quadrada do comprimento dos vetores k ($\sqrt{d^k}$) para garantir maior estabilidade dos gradientes (Figura 11 (g)). Com isso, o resultado de cada par de palavras é submetido à transformação *softmax*, que gera valores no intervalo entre 0 e 1, sendo que a soma de todos os valores é igual a 1 (Figura 11 (h)). Esse resultado representa a “atenção” que a rede deve dar a cada par de palavras. Em seguida, o valor do *softmax* é multiplicado pelo vetor v das palavras correspondentes (ou seja, a palavra com a qual a palavra-foco faz par), resultando em vetores s (Figura 11 (i)), que são somados, produzindo, finalmente, um vetor z para a palavra-foco (Figura 11 (j)), que incorpora as relações entre a palavra-foco e todas as

Figura 11 – Exemplo do cálculo de uma camada de *self-attention*.

Fonte: Própria (2024).

demais palavras da sequência, com a devida atenção considerada (Vaswani et al., 2017).

Todos os valores da Figura 11 são ilustrativos; porém, os vetores s e z estão em diferentes tons para demonstrar que, após a multiplicação dos valores pelo *softmax*, o valor de uma palavra terá maior impacto que o de outra, fazendo com que o vetor z de uma tenha maior peso que o de seu par (neste exemplo, o z_1), demonstrando a relação entre as palavras.

O módulo *encoder* possui um conjunto de matrizes W^q , W^k e W^v , e esse conjunto é denominado de *attention head* (ou cabeça de atenção). A quantidade de *attention heads* é a raiz quadrada do comprimento dos vetores k ($\sqrt{d^k}$). Com isso, o *transformer* realiza o mecanismo de atenção mais de uma vez, resultando, no final da camada de *self-attention*, em $n(\sqrt{d^k})$ vetores z para cada texto, fazendo com que haja atenção em diversos dados (Popel; Bojar, 2018). Esse processo é similar ao que ocorre com o ser humano, que, por exemplo, ao ler um livro, dá atenção a diversas partes do texto ao mesmo tempo, como o nome de um personagem, o ambiente em que se situa, entre outros, e o *transformer* funciona de maneira análoga.

Por fim, a última etapa é concatenar todos os vetores z (*attention heads*) e multiplicá-los por uma matriz de peso W^o , que também é ajustada durante o treinamento do modelo. Cada palavra será representada por um vetor z com o mesmo tamanho do vetor v , e esses vetores passam por uma camada *feed forward*, que irá gerar os *embeddings finais*, os quais são enviados pelo *encoder* para o *decoder*.

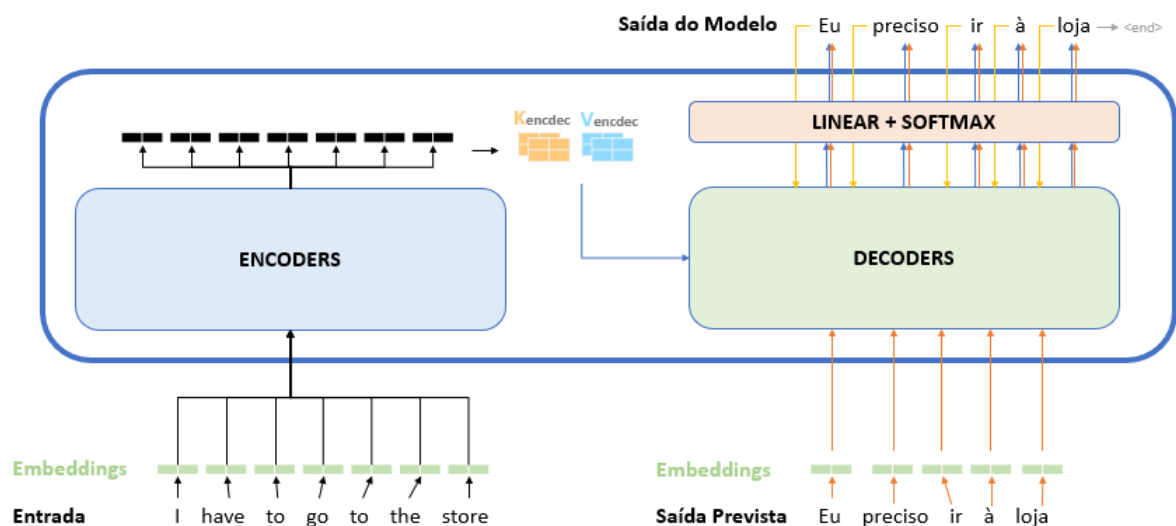
O *decoder* recebe os *embeddings* e transforma-os em matrizes de atenção K e V , que serão aplicadas nas camadas *encoder-decoder attention*. Essa camada aplica a camada *self-attention* no primeiro vetor que recebe, que, no exemplo de tradução, é o início da sequência traduzida, e esse processamento é o mesmo da camada *self-attention* do *encoder*. No entanto, na camada

encoder-decoder attention, são utilizadas as matrizes K e V (que são produtos do processamento da sequência de entrada), ao invés de utilizar as matrizes W^k e W^V .

O resultado da pilha que forma o *decoder* é, da mesma forma que no *encoder*, um *embedding* de comprimento igual ao *embedding* que representa o texto original. Esse *embedding* passa por uma camada linear, convertendo-o em um vetor que, no exemplo de tradução, tem comprimento igual ao do vocabulário do segundo idioma (português), resultando na representação de cada uma das palavras traduzidas.

Esse vetor é ativado pela função *softmax*, resultando em um vetor de probabilidades, e o valor com maior probabilidade indica a palavra que o modelo deve gerar, produzindo, finalmente, uma saída (Figura 12 (seta azul)). Ao retornar essa saída como resultado, a palavra volta para o início do *decoder* (Figura 12 (seta amarela)), para ser utilizada na previsão da próxima palavra, até atingir a última, gerando um *token* que indica que o texto terminou.

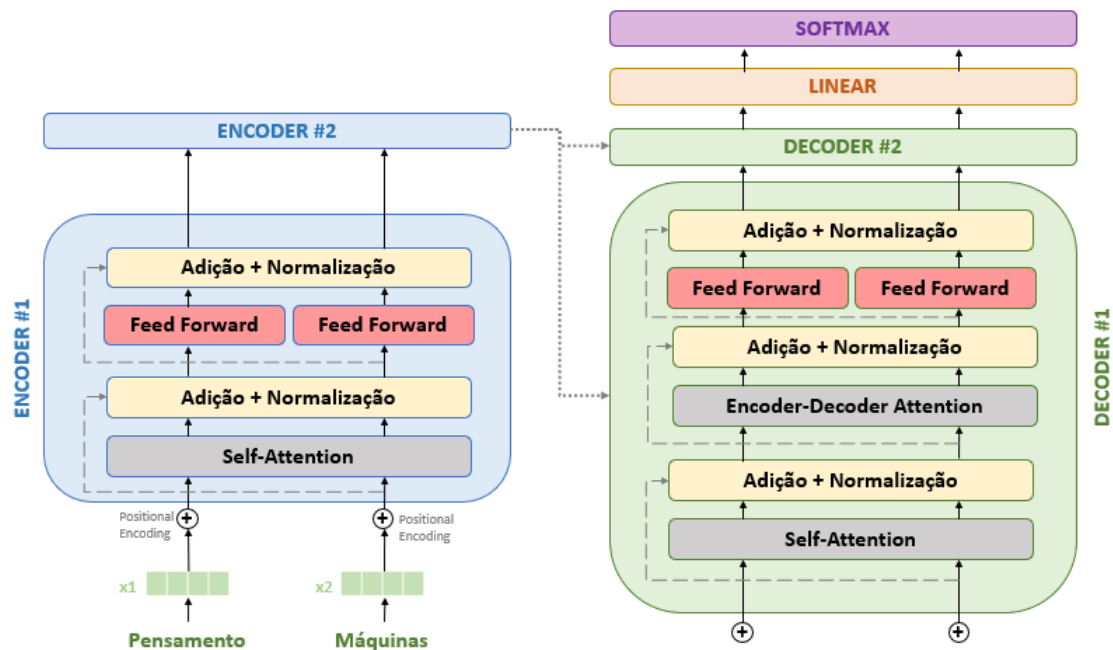
Figura 12 – Representação da pilha (*stack*) do *transformer*.



Fonte: Adaptado de (Alammar, 2018).

Com isso, a arquitetura *transformer* acaba sendo preferida em diversas tarefas em relação a RNNs e LSTMs, pois as palavras não precisam entrar na rede na sequência original; todas elas são processadas simultaneamente, permitindo a paralelização das operações, o que gera um ganho de performance em termos de velocidade. O mecanismo de atenção é aplicado de forma eficiente, criando uma relação entre as palavras, até mesmo aquelas distantes umas das outras, melhorando a performance no quesito precisão (Popel; Bojar, 2018).

Além disso, após cada unidade de atenção e feed-forward, existem camadas de adição e normalização (Popel; Bojar, 2018), e o *transformer* adiciona a cada *embedding* um *positional encoding*, que tem o objetivo de manter a relação temporal entre as palavras, ou seja, a rede sabe qual é a sequência original de cada palavra (Vaswani et al., 2017). A Figura 13 representa as camadas do *encoder* e *decoder* de uma pilha da arquitetura *transformer*.

Figura 13 – Camadas do *encoder* e *decoder* do *transformer*.

Fonte: Adaptado de (Alammar, 2018).

2.7 LLM

Grandes Modelos de Linguagem (*Large Language Models* ou LLMs) são modelos de inteligência artificial que utilizam técnicas de aprendizado de máquina para entender e gerar linguagem humana. Esses modelos se baseiam em redes neurais profundas e adotam técnicas de processamento de linguagem natural para computar e calcular suas respostas (TOUVRON et al., 2023).

As LLMs utilizam um método denominado aprendizado não supervisionado. Nesse processo, o modelo é alimentado com um grande conjunto de dados (bilhões de palavras e frases) com o objetivo de "aprender" sobre a linguagem (TOUVRON et al., 2023).

À medida que o modelo aprende os padrões de associação entre palavras, ele consegue prever estruturas de frases com base na probabilidade. O resultado final é um modelo capaz de capturar as complexas relações entre palavras e frases (TOUVRON et al., 2023).

Com um vasto conjunto de dados e numerosos cálculos probabilísticos, os LLMs exigem um volume significativo de recursos computacionais. As unidades de processamento gráfico (GPUs) são hardwares criados para lidar com tarefas de processamento complexas e simultâneas, sendo ideais para modelos como os LLMs (STRUBELL; GANESH; MCCALLUM, 2019).

Para aprimorar a capacidade de um LLM, a arquitetura Transformer absorve eficientemente as dependências e relações contextuais entre os elementos de uma sequência de dados através de seus parâmetros. Esses parâmetros definem limites que são essenciais para analisar o grande

volume de dados (NAVEED et al., 2024).

Com o desenvolvimento do ChatGPT e do GPT-4 (*Generative Pre-trained Transformer*) pela OpenAI, ocorreu uma revolução na percepção sobre os LLMs (OPENAI et al., 2024). Apesar de possuírem desempenho notável, a OpenAI não divulgou detalhes sobre as estratégias de treinamento ou os parâmetros de peso utilizados.

Em resposta, a Meta desenvolveu o modelo denominado LLaMA (*Large Language Model Meta AI*), que é de código aberto e possui tamanhos variados, de 7 bilhões até 405 bilhões de parâmetros. Os autores afirmam que o modelo apresenta desempenho equivalente aos principais modelos, como o GPT-4, e está próximo de atingir o estado da arte em uma variedade de tarefas (TOUVRON et al., 2023). O LLaMA consiste em duas etapas principais:

- **Pré-treinamento:** no qual o modelo é treinado em grande escala usando tarefas simples como a previsão da próxima palavra (AI@META, 2024).
- **Pós-treinamento:** no qual o modelo é ajustado para seguir instruções, alinhar-se às preferências humanas e melhorar capacidades específicas (como a codificação) (AI@META, 2024).

O modelo LLaMA, atualmente na versão 3.3, suporta nativamente multilinguismo, codificação e raciocínio. Seu maior modelo Transformer possui 405B de parâmetros e processa informações em uma janela de contexto de até 128k tokens (AI@META, 2024).

Para obter esses resultados, os autores afirmam que um grande modelo de linguagem de alta qualidade precisa de:

- **Dados:** Qualidade e quantidade de dados utilizados tanto no pré-treinamento quanto no pós-treinamento são essenciais. Foi utilizado um corpus de cerca de 15T de tokens multilíngues (AI@META, 2024).
- **Escala:** O modelo foi pré-treinado utilizando $3,8 \times 10^{25}$ FLOPs no pré-treinamento. FLOPs ou Operações de Ponto Flutuante representam o número de cálculos de ponto flutuante (adições, multiplicações, etc.) realizados durante o treinamento. A estimativa de FLOPs foi desenvolvida pela OpenAI: $FLOPs = 6ND$, onde N são parâmetros e D são tokens. O LLaMA 3 possui 400B de parâmetros, resultando em $6 \times 400 \times 15T = 3,8 \times 10^{25}$ (AI@META, 2024).
- **Gerenciamento da complexidade:** Maximizar a capacidade de escalar o processo de desenvolvimento do modelo utilizando o modelo Transformer padrão (Vaswani et al., 2017), além de adotar um procedimento de pós-treinamento simples com base em ajuste fino supervisionado (SFT), amostragem de rejeição (RS) e otimização de preferência direta (DPO) (RAFAILOV et al., 2024).

Para a criação do modelo LLaMA 3, o corpus multilíngue foi convertido em tokens e pré-treinado para executar a previsão do próximo token. Nesse estágio, o modelo aprende a estrutura da linguagem (AI@META, 2024).

O pré-treinamento é realizado em grande escala, utilizando um modelo de 405B de parâmetros em 15,6T de tokens, com uma janela de contexto inicial de 8k tokens. Esse processo é sequencial, aumentando progressivamente a janela de contexto até atingir 128k tokens. O modelo utiliza o algoritmo AdamW com uma taxa de aprendizado de pico de 8×10^{-5} , aquecimento linear de 8000 passos e uma programação de taxa de aprendizado decrescente em cosseno até 8×10^{-7} (AI@META, 2024).

Os dados do pré-treinamento incluem informações até o final de 2023 obtidas na web. Foram utilizados métodos de deduplicação e mecanismos de limpeza de dados em cada fonte para garantir tokens de alta qualidade, além de remover domínios com conteúdo adulto e informações pessoalmente identificáveis (AI@META, 2024).

Um classificador foi desenvolvido para categorizar os tipos de informações contidas nos dados, permitindo a inclusão de categorias específicas. Após vários testes, foi estabelecida a seguinte distribuição de tokens: 50% de conhecimento geral, 25% matemáticos e de raciocínio, 17% de código e 8% multilíngues (AI@META, 2024).

No estágio final do pré-treinamento, sequências longas foram utilizadas para dar suporte às janelas de contexto de até 128k tokens. O treinamento em sequências longas foi evitado inicialmente devido ao aumento quadrático na complexidade computacional das camadas de autoatenção. O comprimento do contexto foi aumentado gradualmente até que o modelo se adaptasse com sucesso (AI@META, 2024).

Este processo foi realizado em seis estágios, utilizando aproximadamente 800B de tokens de treinamento. Assim como a OpenAI afirmou, a Meta também relatou que o recozimento melhorou o desempenho dos modelos em até 24%, embora em algumas versões a melhoria tenha sido insignificante. Os autores afirmam que modelos com forte capacidade de aprendizado e raciocínio em contexto não necessitam de amostras específicas de treinamento em domínios para obter bom desempenho (AI@META, 2024).

O LLaMA 3 utiliza a arquitetura Transformer densa e padrão (Vaswani et al., 2017), com atenção de consulta agrupada (AINSLIE et al., 2023) e oito cabeças de chave-valor para melhorar a velocidade de inferência e reduzir o tamanho dos caches durante a decodificação. O modelo também emprega uma máscara de atenção que impede a autoatenção entre diferentes documentos dentro da mesma sequência.

O modelo LLaMA 3 de 405B de parâmetros possui uma arquitetura com 126 camadas e 128 cabeças de atenção, atingindo o tamanho computacionalmente ótimo de acordo com as leis de escala para o orçamento de treinamento da Meta. Essas leis determinam o tamanho ideal do modelo conforme o orçamento computacional (AI@META, 2024).

Após o modelo adquirir uma rica compreensão da linguagem, mas ainda não ser capaz de seguir instruções, inicia-se o pós-treinamento. Este estágio utiliza um modelo de recompensa sobre o ponto de verificação pré-treinado com dados de preferência anotados por humanos. Em seguida, os pontos de verificação são ajustados com Ajuste Fino Supervisionado (SFT) em dados de ajuste de instrução e Otimização de Preferência Direta (DPO) (AI@META, 2024).

O Ajuste Fino Supervisionado (SFT) é uma etapa de treinamento em modelos AM, especialmente usada em LLMs. Essa etapa ocorre após o treinamento inicial (pré-treinamento) do modelo em um grande conjunto de dados não supervisionados. No SFT, o modelo é ajustado usando um conjunto de dados rotulado e supervisionado, onde há exemplos de entrada e saída esperada. O objetivo é alinhar o modelo com tarefas específicas, reduzindo erros e melhorando seu desempenho em problemas delimitados.

A Otimização de Preferência Direta (DPO) é uma técnica emergente que busca alinhar os modelos diretamente com preferências humanas ou critérios de qualidade especificados. Em vez de ajustar o modelo com base em dados supervisionados ou respostas específicas, a DPO utiliza comparações de preferências humanas (como exemplo, a DPO retornará "Resposta A é melhor que Resposta B"). Com base nessas comparações, o modelo é ajustado para produzir respostas que maximizem a preferência do usuário.

O Modelo de Recompensa (RM) utiliza dados de preferência para gerar pares de respostas (escolhida e rejeitada), além de gerar anotações para edição de respostas. Cada amostra de classificação de preferência possui duas ou três respostas com hierarquia clara (editado > escolhido > rejeitado) (AI@META, 2024).

Por fim, o prompt é concatenado com as respostas em uma única linha, embaralhadas aleatoriamente, e então separadas para cálculo de pontuação. Esta abordagem melhora a eficiência do treinamento sem perda de precisão.

Para melhorar ainda mais os modelos de LLM, podem ser utilizadas técnicas como a *Retrieval-Augmented Generation* (RAG). A RAG representa uma inovação significativa na operação dos LLM, pois ajuda a melhorar a precisão e a atualidade das respostas do modelo (LEWIS et al., 2021).

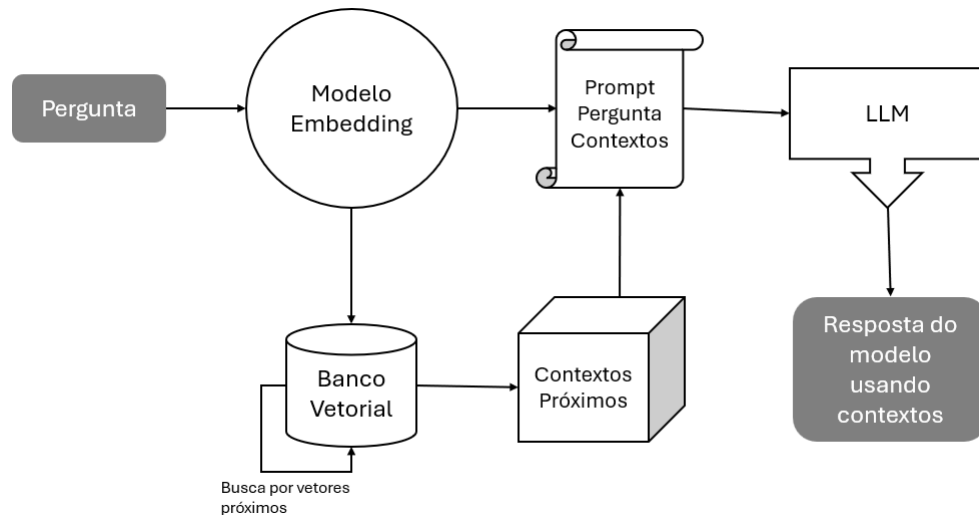
Diferentemente do fine-tuning, a RAG introduz um sistema intermediário de pesquisa em bases de dados externas, frequentemente implementado como Bases de Dados Vetoriais (VecDBs) (AKKIRAJU et al., 2024).

A eficácia da RAG tem sido demonstrada em vários domínios que exigem informações atualizadas. Por exemplo, Gautier Izacard e Touvron, mostraram que modelos RAG superaram os LLM tradicionais em tarefas de pergunta-resposta de domínio aberto, alcançando uma melhoria de até 30% em métricas padrão como *Exact Match* e *F1-score* (IZACARD; GRAVE, 2021).

A metodologia RAG fundamenta-se em três componentes técnicos principais: pesquisa vetorial, similaridade vetorial e *re-ranking*, conforme a Figura 14. Estes elementos permitem que

o sistema supere a simples correspondência de palavras-chave, possibilitando a identificação do significado semântico das palavras para produzir respostas mais contextualizadas (Manning; Raghavan; Schütze, 2008).

Figura 14 – Arquitetura RAG: Geração Aumentada por Recuperação



Fonte: Própria (2024).

- **Pesquisa vetorial:** Utiliza representações numéricas de alta dimensão para codificar o conteúdo semântico dos textos (Mikolov et al., 2013).
- **Similaridade vetorial:** Aplica métricas matemáticas para quantificar a proximidade semântica entre vetores. As métricas mais comuns da similaridade vetorial são: Similaridade do Cosseno, Distância Euclidiana e Produto Escalar (Mikolov et al., 2013).
- **Re-ranking:** Implementa algoritmos adicionais para ajustar os resultados iniciais da pesquisa, dando maior probabilidade aos mais relevantes (Mikolov et al., 2013).

Outra técnica também utilizada para melhorar as respostas de LLMs é a engenharia de *prompt*, que envolve a preparação de estratégias de códigos ou estímulos para gerar respostas características, concisas e proeminentes. Com isso, torna-se uma prática essencial para o aproveitamento máximo das potencialidades dos LLMs (BROWN et al., 2020).

O *prompt* funciona como uma configuração dos modelos para que uma tarefa possa ser concluída. Não se trata apenas da criação de instruções independentes, mas de uma habilidade que tenha um entrosamento profundo das capacidades e limitações dos LLMs. Existem vários estágios para refinar, repetir e testar continuamente os prompts com base nos resultados obtidos (NASCIMENTO, 2024).

O *prompt* deve ser descrito de forma clara e precisa, contendo as informações necessárias para que o modelo compreenda a tarefa a ser executada, bem como o formato da resposta esperada.

Para alcançar melhor qualidade nos dados extraídos, Zhang afirma que, para modelos LLaMa, os prompts devem ser mantidos pequenos e simples para um melhor entendimento da tarefa (ZHANG et al., 2024).

Prompts muito longos podem fazer com que os modelos percam informações relevantes. Além disso, a quantidade de amostras fornecidas como exemplos influencia diretamente na qualidade da extração, pois modelos com mais exemplos claros possibilitam uma compreensão mais precisa de como a tarefa deve ser realizada (ZHANG et al., 2024).

2.8 RASE

Com o objetivo de captar informações presentes em regulamentos, estruturar o processo de conversão dessas regras para uma linguagem computacional e aplicar a verificação automática de modelos em grande escala, Hjelseth desenvolveu as metodologias Tx3 e RASE (HJELSETH, 2015).

A metodologia Tx3 propõe três ações a serem realizadas conforme a classificação das informações presentes nos regulamentos:

- **Transcrever (T1):** Deve ser aplicada para instruções que podem ser diretamente transcritas em regras computáveis;
- **Transformar (T2):** Deve ser aplicada para instruções que necessitam ser reestruturadas ou reescritas de forma a preservar o sentido original;
- **Transferir (T3):** Deve ser aplicada quando os requisitos são expressos de maneira imprecisa, sem conexão clara entre os objetivos do regulamento e os indicadores. Nestes casos, é necessária a análise de um profissional especializado.

Após a classificação do texto original, os elementos classificados como T1 e T2 são reescritos e submetidos à metodologia RASE. A metodologia RASE, baseada em conceitos semânticos, transforma documentos normativos em regras bem definidas, utilizáveis em softwares baseados em BIM (HJELSETH; NISBET, 2011). O processo consiste na identificação de quatro operadores lógicos:

- **Requisito (R):** Expressa o dever associado, frequentemente identificado pelo imperativo “deve”. Por exemplo, na frase “Toda rota acessível de espaço privado deve ser provida de iluminação natural ou artificial com nível mínimo de iluminância de 150 lux medidos a 1,00 m do chão”, o requisito é: “deve ser provida de iluminação natural ou artificial com nível mínimo de iluminância de 150 lux”.
- **Aplicabilidade (A):** Identifica a quem ou ao que o requisito se aplica. No exemplo anterior, a aplicabilidade é: “rota acessível”.

- **Seleção (S):** Indica um subconjunto da aplicabilidade. No exemplo, a seleção é: “de espaço privado”;
- **Exceção (E):** Define os elementos excluídos da regra. Completando o exemplo anterior com a norma “São aceitos níveis inferiores de iluminância para ambientes específicos, como cinemas, teatros ou outros, conforme normas técnicas específicas.”, a exceção é: “níveis inferiores para cinemas, teatros e outros”, pois isso não se aplica a regra.

Para facilitar a visualização, utiliza-se um sistema de cores, onde de costume é utilizado: Requisitos (azul), Aplicabilidade (verde), Seleção (vermelho) e Exceção (laranja). Cada operador identificado é associado a atributos específicos:

- **Objeto:** Termos padronizados, como “escada” ou “corrimão”.
- **Propriedade (A):** Termos classificados, como “nível de iluminância” ou “sistema de segurança”.
- **Comparador:** Pode ser maior, menor, igual, diferente, entre outros.
- **Valor:** Valores numéricos, unidades ou descrições.
- **Unidade:** Tipo de unidade do valor numérico caso exista.

Como exemplo, considere o texto da norma NBR 9050: “A inclinação transversal da superfície deve ser de até 2% para pisos internos e de até 3% para pisos externos. A inclinação longitudinal da superfície deve ser inferior a 5%. Inclinações iguais ou superiores a 5% são consideradas rampas e, portanto, devem atender a 6.6.”

Aplicando a metodologia Tx3 e RASE:

- **Tx3 (Transcrever):**
 - Pisos internos devem ter inclinação transversal menor ou igual a 2%;
 - Pisos externos devem ter inclinação transversal menor ou igual a 3%;
 - Pisos internos devem ter inclinação longitudinal menor do que 5%;
 - Pisos externos devem ter inclinação longitudinal menor do que 5%;
 - Pisos com inclinação longitudinal maior ou igual a 5% são considerados rampas e devem atender a 6.6;
- **RASE (Operadores):**
 - A{Pisos} S{internos} R{devem ter inclinação transversal menor ou igual a 2%};
 - A{Pisos} S{externos} R{devem ter inclinação transversal menor ou igual a 3%};

- A{Pisos} S{internos} R{devem ter inclinação longitudinal menor do que 5%};
- A{Pisos} S{externos} R{devem ter inclinação longitudinal menor do que 5%};
- A{Pisos} E{com inclinação longitudinal maior ou igual a 5% são consideradas rampas} R{devem atender a 6.6};

Por fim, após obter os operadores RASE é gerado o atributo de cada um dos operadores, como mostra a Figura 15.

Figura 15 – Utilização da metodologia RASE na NBR 9050

	Frase Métrica	Operador	Objeto	Propriedade	Comparador	Valor	Und.
Pisos internos devem ter inclinação transversal menor ou igual a 2%.	Pisos	aplicabilidade	componente	tipo	inclui	piso	
	internos	seleção	piso	interno	=	VERDADEIRO	
	devem ter inclinação transversal menor ou igual a 2%.	requisito	piso interno	inclinação transversal	<=	2	%
Pisos externos devem ter inclinação transversal menor ou igual a 3%.	Pisos	aplicabilidade	componente	tipo	inclui	piso	
	externos	seleção	piso	externo	=	FALSO	
	devem ter inclinação transversal menor ou igual a 3%.	requisito	piso externo	inclinação transversal	<=	3	%
Pisos internos e externos devem ter inclinação longitudinal menor do que 5%.	Pisos	aplicabilidade	componente	tipo	inclui	piso	
	internos	seleção	piso	interno	=	VERDADEIRO	
	devem ter inclinação longitudinal menor do que 5%.	requisito	piso interno	inclinação longitudinal	<	5	%
Pisos internos e externos devem ter inclinação longitudinal menor do que 5%.	Pisos	aplicabilidade	componente	tipo	inclui	piso	
	externos	seleção	piso	externo	=	FALSO	
	devem ter inclinação longitudinal menor do que 5%.	requisito	piso externo	inclinação longitudinal	<	5	%
Pisos com inclinação longitudinal maior ou igual a 5% são consideradas rampas devem atender a 6.6.	Pisos	aplicabilidade	componente	tipo	inclui	piso	
	com inclinação longitudinal maior ou igual a 5% são consideradas rampas	exceção	piso	inclinação longitudinal	>=	5	%
	devem atender a 6.6.	requisito	rampa	atender	inclui	Item 6.6 RAMPAS	

Fonte: Própria (2024).

2.9 Trabalhos Relacionados

A integração do BIM com tecnologias de inteligência artificial generativa (IAG), como o ChatGPT, tem sido pouco explorada no setor de Arquitetura, Engenharia e Construção (AEC). Este tema abrange aplicações práticas, frameworks inovadores, desafios e possibilidades futuras, sendo objeto de estudo de diversos trabalhos recentes.

Um dos estudos analisados aborda a integração do BIM com modelos de IAG, como o ChatGPT e o Bard. Este artigo identifica que a utilização dessas tecnologias visa otimizar processos como design e tomada de decisão (RANE; CHOUDHARY; RANE, 2023). No entanto, os autores destacam desafios, como a falta de padronização de dados, deficiências em segurança e a necessidade de capacitação de profissionais do setor (RANE; CHOUDHARY; RANE, 2023). Além disso, os modelos generativos enfrentam dificuldades em interpretar a semântica complexa de dados BIM, além de problemas gerais, como alta demanda computacional e falta de diretrizes éticas (RANE; CHOUDHARY; RANE, 2023). Em resposta a essas lacunas, os pesquisadores propõem um framework voltado à otimização de processos, redução de custos e tempo, e à facilitação da adoção de IA por meio de interfaces mais intuitivas (RANE; CHOUDHARY; RANE, 2023).

Outro estudo relevante trata do desenvolvimento de um chatbot, denominado JULE, projetado para gerentes de obras (LIN, 2023). Este modelo utiliza BIM e técnicas de Processamento de Linguagem Natural (PLN) para fornecer informações precisas e em tempo real no canteiro de obras. O chatbot permite que os usuários consultem dados como dimensões de componentes estruturais e ferramentas necessárias de forma ágil e intuitiva, eliminando a dependência de interfaces convencionais (LIN, 2023). A abordagem proposta demonstrou alta precisão na identificação de intenções e na recuperação de informações essenciais, melhorando a adoção de sistemas baseados em BIM e promovendo maior eficiência no gerenciamento de obras (LIN, 2023).

Um trabalho adicional investigou a aplicação de LLMs, como GPT-3.5 e GPT-4, em projetos BIM voltados à segurança (SAMMOUR et al., 2024). Os autores realizaram testes com 385 questões de múltipla escolha de exames certificados pela BCSP, abrangendo áreas como gestão de sistemas de segurança, identificação de perigos e ergonomia (SAMMOUR et al., 2024). Utilizando técnicas de engenharia de *prompt*, como *Prompt Direct*, *Prompt Chain-of-Thought* e *Prompt Few-Shot*, os modelos foram avaliados em termos de precisão e consistência. Os resultados indicaram que o GPT-4 alcançou 84,6% de precisão, enquanto o GPT-3.5 obteve 73,8%. Apesar do bom desempenho em áreas como gerenciamento de sistemas e controle de perigos, ambos os modelos apresentaram dificuldades em responder questões relacionadas a emergências e cálculos matemáticos (SAMMOUR et al., 2024). A pesquisa destacou o potencial desses modelos, mas também a necessidade de melhorias para evitar informações imprecisas ou enviesadas (SAMMOUR et al., 2024).

Por fim, outro estudo explorou diferentes técnicas de engenharia de prompt para otimizar a utilização de LLMs no setor AEC. Os autores identificaram que métodos genéricos de interação limitam a eficácia desses modelos, resultando em saídas desalinhadas com as necessidades do setor (SAMSAMI, 2024). Problemas como confiabilidade, adaptação a regulamentações e aceitação no setor também foram identificados como barreiras. A pesquisa coletou técnicas de prompt eficazes, como prompts instrucionais e baseados em restrições, e testou essas abordagens em projetos reais (SAMSAMI, 2024). Os resultados mostraram que prompts específicos reduziram erros em cronogramas e melhoraram o reconhecimento de perigos. Apesar disso, limitações como alucinações do modelo para tarefas complexas e a necessidade de validação externa ainda persistem (SAMSAMI, 2024).

Os trabalhos analisados fornecem uma visão abrangente sobre o estado da arte na integração de LLMs e BIM no setor AEC. Embora avanços significativos tenham sido alcançados, desafios importantes, como a interpretação semântica de dados BIM, segurança e a adaptação de LLMs às demandas específicas do setor, ainda necessitam de soluções mais robustas. Assim, este campo continua a ser promissor para futuras pesquisas e inovações.

2.9.1 Análise dos Trabalhos Relacionados

A integração de BIM com LLMs e IA generativa no setor AEC tem mostrado avanços, mas também apresenta desafios. Um estudo identificou a utilização de tecnologias como ChatGPT e Bard para otimizar processos, como design e tomada de decisão. No entanto, as dificuldades incluem a falta de padronização de dados, questões de segurança e a complexidade semântica dos dados BIM (RANE; CHOUDHARY; RANE, 2023). O uso de LLMs aqui é bom devido à sua capacidade de processar grandes volumes de dados e fornecer *insights* valiosos, mas a interpretação semântica continua sendo uma limitação.

Outro estudo sobre o chatbot JULE, que usa BIM e PLN, mostrou avanços na eficiência do gerenciamento de obras, permitindo consultas rápidas no canteiro. No entanto, como o modelo depende de dados BIM bem estruturados, as falhas na semântica e na adaptação dos dados para o modelo ainda representam uma barreira (LIN, 2023). Mesmo assim, o uso de LLMs para interpretar regras e padrões regulatórios é promissor devido à sua agilidade na recuperação de informações.

Um estudo com LLMs como GPT-3.5 e GPT-4 aplicados a segurança em BIM demonstrou precisão significativa, mas também dificuldades em questões complexas como emergências e cálculos. A limitação aqui está na falta de confiabilidade para questões mais técnicas, e embora os LLMs possam interpretar normas e regulamentos, é necessário garantir maior precisão e evitar informações enviesadas (SAMMOUR et al., 2024).

Além disso, a pesquisa sobre técnicas de engenharia de prompt para otimizar LLMs no setor AEC destacou que prompts genéricos não são eficazes. O uso de prompts mais específicos

melhorou a precisão, mas problemas como alucinações do modelo e a necessidade de validação externa persistem ([SAMSAMI, 2024](#)). A adaptação dos modelos às regulamentações específicas do setor é crucial, e o uso de LLMs para interpretar regras regulatórias se mostra útil, mas ainda exige ajustes finos e validação.

3

Proposta

A pesquisa propõe o desenvolvimento de um modelo LLM baseado no LLaMa 3, open-source, com o objetivo de interpretar regras e padrões regulatórios da AEC e convertê-los para a metodologia RASE. O sistema converterá essas informações para o software Revit, implementando um código de validação para verificar se os elementos do projeto estão em conformidade com as normas, destacando os corretos e incorretos. Para treinar o modelo, serão usadas bases de dados que convertem normas regulatórias em código RASE. O autor Eduardo ([MENDONÇA](#); [MANZIONE, 2020](#)) é uma referência para essa abordagem. Para alcançar o resultado esperado, serão realizadas três tarefas distintas com três modelos de treinamento diferentes, sendo um processo de conversão das normas para dados estruturados RASE.

3.1 Materiais e Métodos

A metodologia proposta para este trabalho busca construir um conjunto de modelos utilizando o LLaMa 3 para interpretar normas AEC e convertê-las para a linguagem RASE. A conversão será realizada em três etapas, com cada etapa correspondendo a um modelo específico, que será treinado e ajustado de acordo com técnicas de engenharia de prompt e estratégias de treinamento adequadas.

Os modelos utilizados são:

- **Modelo R1 - Extração de Normas e Separação de Regras:**
 - **Objetivo:** Identificar e extrair as normas presentes em documentos regulatórios da AEC e dividi-las em regras individuais.
 - **Modelo:** Modelo R1.
 - **Entrada:** Documento normativo (exemplo: regulamento de construção, código de edificações).

- **Saída:** Regras separadas, com cada norma identificada como uma unidade de verificação.
 - **Exemplo de Entrada:** "A inclinação transversal da superfície deve ser de até 2% para pisos internos e de até 3% para pisos externos."
 - **Exemplo de Saída:** ["Pisos internos devem ter inclinação transversal menor ou igual a 2%;", "Pisos externos devem ter inclinação transversal menor ou igual a 3%;"]
- **Modelo R2 - Organização das Regras em Operadores RASE:**
 - **Objetivo:** Classificar as regras extraídas de acordo com os quatro atributos do modelo RASE: Requisito, Aplicabilidade, Seleção e Exceção.
 - **Modelo:** Modelo R2.
 - **Entrada:** Regras extraídas na etapa anterior.
 - **Saída:** Regras organizadas em operadores RASE, permitindo a análise semântica de sua aplicação.
 - **Exemplo de Entrada:** "Pisos internos devem ter inclinação transversal menor ou igual a 2%;"
 - **Exemplo de Saída:** "A{Pisos} S{internos} R{devem ter inclinação transversal menor ou igual a 2%};"
 - **Modelo R3:**
 - **Objetivo:** Converter os operadores identificados na etapa anterior em atributos RASE aplicáveis, ou seja, regras que podem ser implementadas diretamente em softwares de modelagem (BIM).
 - **Modelo:** Modelo R3.
 - **Entrada:** Operadores RASE organizados.
 - **Saída:** Atributos RASE em JSON, prontos para verificação em ferramentas de modelagem de projetos, como o Revit.
 - **Exemplo de Entrada:**

```
1 {
2   "texto_completo": "Os corrimãos podem ser acoplados aos
      guarda-corpos.",
3   "texto_segmentado": "Os A{corrimãos} R{podem ser
      acoplados aos guarda-corpos.}",
4 }
```
 - **Exemplo de Saída:**

```
1 {
2   "texto_completo": "Os corrimaos podem ser acoplados aos
3     guarda-corpos.",
4   "texto_segmentado": "Os A{corrimaos} R{podem ser
5     acoplados aos guarda-corpos.}",
6   "segmentos": [
7     {
8       "texto_metrico": "corrimaos",
9       "tipo": "aplicabilidade",
10      "objeto": "componente",
11      "propriedade": "tipo",
12      "comparacao": "=",
13      "valor": "corrimao",
14      "unidade": ""
15    },
16    {
17      "texto_metrico": "podem ser acoplados aos
18        guarda-corpos.",
19      "tipo": "requisito",
20      "objeto": "corrimao",
21      "propriedade": "guarda-corpo",
22      "comparacao": "=",
23      "valor": "VERDADEIRO",
24      "unidade": ""
25    }
26  ]
27 }
```

Para alcançar os objetivos propostos, a pesquisa utilizará as seguintes ferramentas e bases de dados:

- **LLaMa 3:** Modelo base para as três etapas de transformação das normas.
- **PostgresVector:** Usado para armazenamento e consulta de dados vetoriais, facilitando a busca por similaridade em grandes volumes de texto. A consulta será feita por similaridade de cosseno para otimizar a recuperação de informações relevantes.
- **Base de Dados de Normas AEC:** Dados específicos para cada modelo, baseados no trabalho de Eduardo ([MENDONÇA; MANZIONE, 2020](#)), que serão expandidos para melhorar o desempenho, especialmente no uso das técnicas mais avançadas. Caso seja

necessário, utilizará de dados em inglês e convertidos para linguagem em português a partir de uma LLM como ChatGPT 4.

Cada modelo empregará técnicas de engenharia de prompt para otimizar os resultados dos LLMs, minimizando possíveis alucinações. Serão utilizados prompts específicos desenvolvidos pelo autor Samsami ([SAMSAMI, 2024](#)), Brown ([BROWN et al., 2020](#)) e Radford ([RADFORD; WU; AMODEI, 2019](#)), que já demonstraram eficácia em reduzir erros em áreas específicas, com a expectativa de auxiliar também na conversão de normas AEC para RASE. Os tipos de engenharia de prompt que podem ser úteis são:

- **Prompt Direct:**

- **Descrição:** Essa técnica envolve a formulação de um prompt claro e direto, que instrui o modelo a realizar uma tarefa específica sem ambiguidade. O objetivo aqui é fornecer ao modelo uma solicitação precisa para que ele execute a tarefa de maneira eficaz, sem sobrecarregar o modelo com informações desnecessárias ou erradas ([SAMSAMI, 2024](#)). Em seu caso, isso pode ser utilizado para solicitar a segmentação de uma norma AEC ou a aplicação de operadores RASE de forma simples e objetiva.
- **Exemplo:** "Divida a norma a seguir em segmentos de aplicabilidade, requisito e seleção: 'Os corrimãos podem ser acoplados aos guarda-corpos e devem ser construídos com materiais rígidos.'"
- **Eficácia:** Acredita que essa abordagem seja a mais eficaz para o **Modelo R1**, que tem como objetivo a segmentação clara das normas.

- **Prompt Chain-of-Thought:**

- **Descrição:** Essa técnica envolve pedir ao modelo para seguir um raciocínio passo a passo, ou seja, encorajá-lo a fazer um "raciocínio explícito" sobre a tarefa. Ao invés de simplesmente gerar uma resposta, o modelo deve explicar suas decisões ou como chegou a um resultado ([BROWN et al., 2020](#)). Esse tipo de engenharia de prompt é útil quando a tarefa exige uma cadeia de raciocínios complexos, como a atribuição de operadores RASE em um texto normativo.
- **Exemplo:** "Primeiro, identifique se a sentença envolve um requisito, uma aplicabilidade ou uma seleção. Explique o porquê, e depois aplique os operadores RASE de acordo."
- **Eficácia:** Acredita que essa abordagem seja mais adequada para o **Modelo R2**, que requer a aplicação de operadores RASE aos segmentos gerados pelo Modelo R1.

- **Prompt Few-Shot:**

- **Descrição:** Essa técnica envolve fornecer ao modelo exemplos de entrada e saída antes de fazer a solicitação principal. Ao fornecer um número limitado de exemplos de como as normas devem ser convertidas para a linguagem RASE, o modelo aprende a generalizar essas tarefas com menos dados (RADFORD; WU; AMODEI, 2019). Essa abordagem é ideal quando há uma quantidade limitada de dados rotulados ou quando o modelo precisa aprender a partir de um número reduzido de exemplos, como no caso das normas AEC, que podem ser complexas e variadas.
- **Exemplo:** 'As portas devem ter uma altura mínima de 2 metros.' → 'A{portas} R{devem ter uma altura mínima de 2 metros.}' Agora, converta a seguinte norma: 'Os corrimãos devem ser feitos de materiais rígidos.'"
- **Eficácia:** Acredita que esse método seja mais adequado para o **Modelo R2** e **Modelo R3**, que transforma os segmentos em atributos RASE no formato JSON.

Para otimizar o desempenho dos modelos em cada uma das etapas de conversão das normas para a linguagem RASE, serão aplicadas quatro abordagens, todas utilizando engenharia de prompt para ajustar o comportamento dos modelos LLaMa às necessidades específicas do problema:

- **Abordagem EP:** Esta abordagem utiliza exclusivamente engenharia de prompt, empregando prompts específicos e cuidadosamente elaborados para garantir que os modelos sejam orientados a realizar tarefas de segmentação de normas de forma eficaz e precisa. A engenharia de prompt é aplicada aqui para otimizar a interpretação e processamento das normas AEC.
- **Técnica EPFT:** Além da engenharia de prompt, esta técnica incorpora fine-tuning do modelo, utilizando uma base de dados orientada ao problema de conversão de normas. O objetivo do fine-tuning é ajustar os parâmetros do modelo com base em dados específicos, aprimorando sua performance na conversão de normas para os operadores RASE.
- **Técnica EPRAG:** A aplicação de engenharia de prompt é combinada com um banco de dados vetorial, que serve para fornecer contexto adicional e melhorar a capacidade do modelo em interpretar corretamente as normas. Esse contexto adicional ajudará os modelos a fornecer respostas mais precisas, especialmente quando envolvem complexidade semântica, como no caso de normas com várias condições e exceções.
- **Técnica EPFTRAG:** Esta abordagem é uma combinação das técnicas EPFT e EPRAG, utilizando tanto fine-tuning quanto contexto adicional via banco de dados vetorial. Esta técnica será aplicada principalmente ao Modelo R3, que exige uma compreensão mais profunda das normas para gerar atributos RASE no formato JSON, com base em uma maior quantidade de contexto.

Os parâmetros padrão (AI@META, 2024) do LLaMa para o fine-tuning incluem uma taxa de aprendizado de $2e-5$, um tamanho de lote de 32, e 3 épocas de treinamento, com o otimizador AdamW e taxa de decaimento do peso de 0,01. Esses parâmetros podem ser ajustados conforme necessário, dependendo da capacidade computacional.

Cada modelo será alimentado com conjuntos de dados específicos, todos os dados são de normas brasileiras e cada regra irá obter as transformações (respostas) para todos os modelos (R1, R2 e R3). A avaliação dos resultados será feita manualmente, com base na precisão das respostas geradas em relação ao conteúdo normativo correto. O uso de dados no formato JSON para o Modelo R3 garantirá que os atributos sejam corretamente gerados para validação em projetos.

A comparação entre as abordagens será feita com base em três métricas principais:

- **Precisão:** A exatidão na interpretação e conversão das normas para o formato RASE.
- **Eficiência:** A rapidez e eficácia na transformação das normas em dados estruturados para implementação no software BIM.
- **Generalização:** A capacidade do modelo de lidar com diferentes conjuntos de dados de normas e sua adaptação a novas regras ou documentos regulatórios.

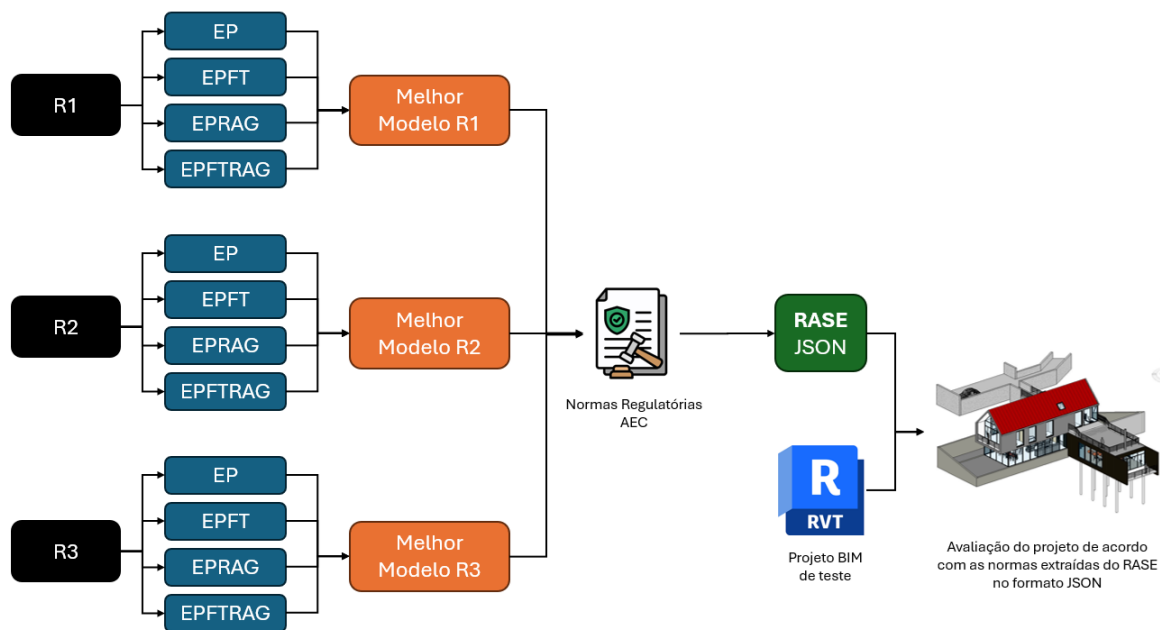
Após o treinamento e avaliação dos modelos, as normas AEC convertidas para RASE serão integradas ao Revit por meio da extensão Dynamo. O código gerado fará a validação dos elementos do projeto, destacando os componentes corretos em verde, incorretos em vermelho, e sem regras associadas em azul, como mostra a Figura 16. Além disso, uma planilha será gerada para documentar os resultados da avaliação.

Para cada projeto selecionado, serão identificados todos os elementos que estão dentro, fora ou não associados às regras selecionadas. Após a identificação, as informações do RASE serão processadas e a precisão do JSON será analisada.

Por fim, serão analisadas três questões a partir dos dados avaliados:

1. Qual abordagem de engenharia de prompt (EP, EPFT, EPRAG, EPFTRAG) proporciona os melhores resultados em termos de precisão e eficiência para converter normas em dados RASE?
2. Os modelos gerados conseguem aplicar as normas de forma eficaz no software Revit, validando corretamente os elementos dos projetos?
3. Qual é a capacidade dos modelos em generalizar para novos conjuntos de dados normativos e diferentes tipos de regras?

Figura 16 – Pipeline da pesquisa.



Fonte: Própria (2024).

Essas questões ajudarão a determinar se o objetivo da pesquisa — automatizar a validação de projetos BIM de acordo com as normas AEC — foi alcançado com sucesso, garantindo precisão na conversão das normas em dados estruturados e aplicáveis.

3.2 Desenvolvimento Inicial

No desenvolvimento inicial da pesquisa, foram implementados algoritmos baseados em três modelos LLaMa 3 (R1, R2 e R3), com o objetivo de converter normas regulatórias AEC para a linguagem RASE. Para esta fase inicial, foi utilizada uma abordagem de engenharia de prompt com prompts genéricos, para avaliar a capacidade dos modelos de interpretar as normas de forma básica e aplicarem os operadores RASE.

Foram realizados cerca de 30 testes iniciais para validar o desempenho dos modelos com os prompts genéricos. Cada modelo foi alimentado com documentos normativos e, após a aplicação dos prompts, os resultados foram avaliados quanto à sua precisão na extração, organização e conversão das normas para RASE. A avaliação foi feita de forma manual, lendo-se um a um e analisando-se os resultados.

Os modelos, de forma geral, demonstraram um desempenho em torno de 90% nas tarefas de segmentação (R1) e organização de regras (R2). No entanto, o Modelo R3 teve dificuldades na conversão de alguns atributos para o formato JSON, especialmente devido à falta de implementação de atributos padrão do RASE. Por exemplo, quando a norma indicava "nível de iluminação", o modelo retornou apenas "luz", o que sugere que o modelo não possuía a base

de dados completa de atributos RASE.

Os algoritmos foram implementados em Python utilizando a biblioteca Langchain, que irá facilitar na integração com bancos de dados vetoriais. Essa biblioteca será especialmente útil quando se utilizar o RAG ou técnicas de engenharia de prompt. A base de dados utilizada foi a disponibilizada pelo autor Eduardo ([MENDONÇA; MANZIONE, 2020](#)), no qual, o mesmo inseriu dados que podem ser utilizados para os três modelos (R1, R2 e R3) na abordagem EP, porém não são dados suficientes para realizar as outras abordagens (EPFT, EPRAG, EPFTRAG).

3.3 Desafios Futuros

Apesar dos bons resultados iniciais, alguns desafios importantes permanecem para o próximo estágio do desenvolvimento. Estes desafios estão principalmente relacionados à melhoria da conversão de atributos no Modelo R3, que será essencial para garantir a aplicação eficaz dos atributos RASE no Revit.

- **Implementação de Atributos Padrão RASE:** O modelo R3 precisa ser ajustado para considerar atributos padrão não identificados nos testes iniciais, como "nível de iluminação", "dimensões mínimas", entre outros. Para isso, será necessário criar uma base de dados de atributos RASE padrão ou realizar o fine-tuning do modelo com dados que incluam esses atributos. Isso permitirá que o modelo faça conversões mais precisas e confiáveis para o formato JSON, atendendo a uma gama mais ampla de normas.
- **Aprimoramento da Qualidade dos Prompts:** Embora os prompts genéricos tenham funcionado bem, será necessário desenvolver prompts mais específicos e contextualmente adaptados para cada modelo (R1, R2 e R3). A criação de prompts mais refinados será fundamental para garantir que o modelo compreenda com precisão as nuances das regulamentações e as converta corretamente para os operadores e atributos RASE. Esse refinamento contínuo dos prompts deve ser uma prioridade para melhorar o desempenho do sistema.
- **Ajuste de Parâmetros do Fine-Tuning:** Para melhorar a conversão para atributos RASE, será necessário realizar fine-tuning no modelo utilizando um conjunto de dados mais robusto e focado em normas AEC com exemplos mais variados. A calibração dos parâmetros de fine-tuning, como a taxa de aprendizado e o número de épocas, será essencial para garantir que o modelo se adapte bem aos requisitos específicos das normas. Com um número maior de exemplos e uma abordagem mais direcionada, o modelo pode alcançar maior precisão na conversão das normas.
- **Utilização de Banco de Dados Vetorial:** Para aprimorar a precisão da conversão das normas e facilitar a recuperação de informações relevantes durante o processo de treinamento,

será implementado um banco de dados vetorial, como o PostgresVector. Esse banco de dados permitirá armazenar as normas e atributos RASE em formato vetorial, otimizando a busca por similaridade através de consultas baseadas em cosseno. A utilização de um banco de dados vetorial proporcionará um acesso mais eficiente a grandes volumes de dados normativos, possibilitando ao modelo uma melhor generalização e uma resposta mais precisa, especialmente quando houver complexidade semântica nas regras.

- **Validação e Integração com Revit:** Após a implementação dos ajustes nos modelos, será necessário validar a integração com o software Revit, usando a extensão Dynamo para verificar se os atributos RASE gerados estão sendo corretamente aplicados aos elementos do projeto. Isso envolve garantir que as normas sejam interpretadas de maneira que resultem em validações precisas e eficientes para os projetos de arquitetura e engenharia.
- **Aumento da Base de Dados:** A base de dados de normas AEC deve ser expandida para incluir uma variedade maior de documentos regulatórios, cobrindo diferentes regiões, tipos de construção e novas regulamentações que possam surgir. A diversidade e o volume maior de dados permitirão que o modelo aprenda a generalizar melhor as normas, lidando com diferentes formatos e interpretações. Essa expansão será fundamental para treinar os modelos de forma mais robusta e ampliar sua aplicabilidade a uma gama mais ampla de normas.
- **Aumento de Testes:** O número de testes realizados precisa ser ampliado para garantir que os modelos se comportem de maneira consistente e eficaz em diferentes cenários. Testes adicionais devem ser realizados para avaliar a precisão da segmentação das normas, a organização dos operadores RASE e a conversão para o formato JSON, cobrindo um número maior de exemplos e cenários variados. Essa ampliação de testes ajudará a identificar eventuais falhas ou limitações no modelo que ainda não foram detectadas nos experimentos iniciais e permitirá ajustes finos antes da implementação final.

Esses desafios são cruciais para a continuidade do desenvolvimento e garantir que o sistema final seja robusto e capaz de lidar com uma variedade de documentos normativos da AEC, promovendo a conformidade e eficiência nos projetos de construção. A combinação de uma base de dados mais ampla, o refinamento contínuo dos testes e a adaptação dos modelos às regulamentações e padrões específicos será essencial para alcançar os objetivos da pesquisa.

3.4 Fases da projeto e cronograma

Os desafios definidos na seção *Desafios Futuros* são o objetivo central de estudo deste mestrado. Através deles, serão delineadas as fases que, por meio da resolução desses desafios, levarão ao alcance dos objetivos da pesquisa. Cada fase será essencial para aprimorar as

ferramentas e técnicas utilizadas no projeto, possibilitando uma solução robusta para a conversão de normas AEC em dados estruturados e aplicáveis no contexto BIM.

O cronograma será executado em 12 meses, como mostra na Figura 17, divididos em etapas:

- **Coleta e Preparação de Dados (Meses 1-2):** Nessa fase, será feita a coleta de dados necessários para o projeto, com foco em dados que irão compor o banco de dados vetorial. Também incluirá a preparação dos dados para treinamento e ajustes no modelo (fine-tuning). Exemplos de prompt também serão elaborados para validar os modelos. O desafio dessa fase é a **Implementação de Atributos Padrão RASE**.
- **Criação, Teste e Avaliação de Prompts (Mês 3):** Durante este mês, o foco será na criação de prompts específicos (*Prompt Few-Shot*, *Prompt Chain-of-Thought*, *Prompt Direct*) para cada modelo, bem como a execução de testes para avaliar a eficácia de cada prompt. A avaliação será feita com base no desempenho dos modelos ao utilizarem os prompts criados. O desafio dessa fase é a **Aprimoramento da Qualidade dos Prompts**.
- **Implementação de Fine-tuning (Mês 4):** Nesta fase, o fine-tuning dos modelos será implementado de forma mais detalhada. Após essa implementação, cada modelo será avaliado novamente para medir seu desempenho após o ajuste. O desafio dessa fase é a **Ajuste de Parâmetros do Fine-Tuning**.
- **Implementação do RAG (Mês 5):** O RAG (contexto vetorial) será implementado em cada um dos modelos para melhorar a contextualização e o desempenho. Após a implementação, uma nova avaliação será feita para verificar os efeitos do RAG. O desafio dessa fase é a **Utilização de Banco de Dados Vetorial**.
- **Implementação da Técnica EPFTRAG (Mês 6):** A técnica EPFTRAG será implementada, com avaliação de como ela impacta cada modelo. O objetivo é testar essa abordagem específica para otimizar os resultados dos modelos. O desafio dessa fase é a **Aumento da Base de Dados e Aumento de Testes**.
- **Implementação no Revit® (Meses 7-8):** Durante esses meses, os resultados dos melhores modelos do projeto serão implementados no software Revit®, focando na integração prática dos modelos e técnicas desenvolvidas no contexto da ferramenta de modelagem. O desafio dessa fase é a **Validação e Integração com Revit**.
- **Escrita do Artigo (Meses 9-10):** Com o trabalho de pesquisa avançado, a escrita do artigo científico será realizada. Este artigo resumirá os resultados obtidos ao longo do projeto e sua contribuição para a área.

- **Escrita da Dissertação (Meses 10-12):** Após o artigo, a dissertação será escrita, detalhando todos os aspectos do projeto, metodologia, resultados e discussões, com um foco mais aprofundado para a defesa acadêmica.
- **Apresentação da Defesa (Mês 12):** Finalmente, o projeto será concluído com a apresentação da defesa, onde os resultados do trabalho serão apresentados e defendidos diante da banca examinadora.

Essas são as fases que guiarão o cronograma do projeto (Figura 17), cada uma com seu objetivo e entregas específicas.

Figura 17 – Cronograma de execução

Tarefas	Meses											
	1	2	3	4	5	6	7	8	9	10	11	12
Coleta e Preparação de Dados	x	x										
Criação, Teste e Avaliação de Prompts			x									
Implementação de Fine-tuning				x								
Implementação do RAG					x							
Implementação da Técnica EPFTRAG						x						
Implementação no Revit®							x	x				
Escrita do Artigo									x	x		
Escrita da Dissertação										x	x	x
Apresentação da Defesa												x

Fonte: Própria (2024).

Referências

- Abdel-Nasser, M.; Mahmoud, K. Accurate photovoltaic power forecasting models using deep lstm-rnn. *Neural Computing & Applications*, Springer London, p. 2727–2740, jul 2019. ISSN 0941-0643. Citado na página 29.
- AI@META. Llama 3 model card. *github*, 2024. Disponível em: <https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md>. Citado 5 vezes nas páginas 10, 37, 38, 39 e 52.
- AINSLIE, J. et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. 2023. Disponível em: <<https://arxiv.org/pdf/2305.13245v3>>. Citado na página 38.
- AKKIRAJU, R. et al. *FACTS About Building Retrieval Augmented Generation-based Chatbots*. 2024. Disponível em: <<https://arxiv.org/abs/2407.07858>>. Citado na página 39.
- Alammar, J. *The Illustrated Transformer*. 2018. Acesso em: 15/05/2021. Disponível em: <<http://jalammar.github.io/illustrated-transformer/>>. Citado 2 vezes nas páginas 35 e 36.
- ALAWADHI, M.; YAN, W. Deep learning from parametrically generated virtual buildings for real-world object recognition. *arXiv preprint arXiv:2302.05283*, 2023. Disponível em: <<https://arxiv.org/abs/2302.05283>>. Citado na página 10.
- Arroyo-Figueroa, G.; Sucar, L. E.; Villavicencio, A. Fuzzy intelligent system for the operation of fossil power plants. *Engineering Applications of Artificial Intelligence*, v. 13, n. 4, p. 431–439, 2000. ISSN 0952-1976. Citado na página 18.
- Bahdanau, D.; Cho, K. H.; Bengio, Y. Neural machine translation by jointly learning to align and translate. In: *3rd International Conference on Learning Representations, ICLR 2015 ; Conference date: 07-05-2015 Through 09-05-2015*. [S.l.: s.n.], 2015. Citado na página 32.
- Bengio, Y. et al. A neural probabilistic language model. *J. Mach. Learn. Res.*, JMLR.org, v. 3, p. 1137–1155, mar 2003. ISSN 1532-4435. Citado na página 21.
- BOTTEGA, B. S. *Avaliação dos efeitos do uso da tecnologia BIM sobre a coordenação de projetistas*. [S.l.]: Dissertação de Mestrado - Universidade Federal do Rio Grande do Sul, 2012. Citado na página 14.
- Braga, A. de P.; Ludermir, T. B.; Carvalho, A. C. P. de L. F. *Redes neurais artificiais: teoria e aplicações*. [S.l.]: Ltc, 2000. Citado na página 27.
- BROWN, T. B. et al. *Language Models are Few-Shot Learners*. 2020. Disponível em: <<https://arxiv.org/abs/2005.14165>>. Citado 2 vezes nas páginas 40 e 50.
- Bulegon, H.; Moro, C. M. C. Mineração de texto e o processamento de linguagem natural em sumários de alta hospitalar. In: *Mineração de texto e o processamento de linguagem natural em sumários de alta hospitalar*. SP - Brasil: Journal of Health Informatics, 2010. v. 2, n. 2. Citado na página 19.
- Cerri, R.; Carvalho, A. C. P. de Leon Ferreira de. Aprendizado de máquina: Breve introdução e aplicações. *Cadernos de Ciência & Tecnologia*, Brasília, v. 34, n. 3, p. 297–313, 12 2017. Citado na página 16.

Chai, T.; Draxler, R. Root mean square error (rmse) or mean absolute error (mae)? – arguments against avoiding rmse in the literature. *Geoscientific Model Development*, v. 7, n. 3, p. 1247–1250, 2014. Citado na página 26.

Chung, J. et al. *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*. 2014. Citado na página 30.

COELHO, S. S.; NOVAES, C. C. Modelagem de informações para construção (bim) e ambientes colaborativos para gestão de projetos na construção civil. *Exatas e Engenharia*, v. 8, n. 23, 12 2018. Citado 2 vezes nas páginas 13 e 14.

Conneau, A. et al. Supervised learning of universal sentence representations from natural language inference data. In: *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. Copenhagen, Denmark: Association for Computational Linguistics, 2017. p. 670–680. Citado na página 21.

DAVENPORT, T. H.; PATIL, D. Data scientist: the sexiest job of the 21st century. *Harvard Business Review*, v. 1, n. 1, 2012. Disponível em: <<https://hbr.org/2012/10/data-scientist-the-sexiest-job-of-the-21st-century>>. Citado na página 9.

Devlin, J. et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. Citado na página 21.

DU, C.; NOUSIAS, S.; BORRMANN, A. Towards a copilot in bim authoring tool using a large language model-based agent for intelligent human-machine interaction. *arXiv preprint arXiv:2406.16903*, 2024. Disponível em: <<https://arxiv.org/abs/2406.16903>>. Citado na página 11.

EASTMAN, C.; TEICHOLZ, P.; SACKS, R. *Bim Handbook: A Guide to Building Information Modeling for Owners, Managers, Designers, Engineers and Contractors*. 2. ed. Nova Jersey: Wiley, 2011. ISBN 978-04-7054-137-01. Citado 3 vezes nas páginas 9, 13 e 14.

Faceli, K. et al. *Inteligência Artificial - Uma Abordagem de Aprendizado de Máquina*. 1. ed. Rio de Janeiro: Ltc, 2011. 192 p. ISBN 9788521618805. Citado na página 15.

Feng, J.; Lu, S. Performance analysis of various activation functions in artificial neural networks. *Journal of Physics: Conference Series*, v. 1237, p. 22–30, 06 2019. Citado 2 vezes nas páginas 25 e 26.

FUCHS, S. et al. Using large language models for the interpretation of building regulations. *arXiv preprint arXiv:2407.21060*, 2024. Disponível em: <<https://arxiv.org/abs/2407.21060>>. Citado 2 vezes nas páginas 10 e 11.

GAO, Y. et al. Retrieval-augmented generation for large language models: A survey. *ArXiv*, abs/2312.10997, 2023. Disponível em: <<https://api.semanticscholar.org/CorpusID:266359151>>. Citado na página 11.

Goodfellow, I.; Bengio, Y.; Courville, A. *Deep Learning*. [S.l.]: MIT Press, 2016. 166–223 p. Citado 3 vezes nas páginas 23, 25 e 28.

Graves, A.; Jaitly, N.; Mohamed, A. rahman. Hybrid speech recognition with deep bidirectional lstm. In: *2013 IEEE Workshop on Automatic Speech Recognition and Understanding*. [S.l.: s.n.], 2013. p. 273–278. Citado na página 30.

Guimarães, L. M. S.; Meireles, M. R. G.; Almeida, P. E. M. de. Avaliação das etapas de pré-processamento e de treinamento em algoritmos de classificação de textos no contexto da recuperação da informação. *Perspectivas em Ciência da Informação*, Belo Horizonte, v. 24, n. 1, p. 169–190, 05 2019. Citado na página 16.

GUO, Y. et al. Spottune: Transfer learning through adaptive fine-tuning. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. Disponível em: <<https://ieeexplore.ieee.org/document/8953749/>>. Citado na página 11.

Hafemann, L. G. *An Analysis of Deep Neural Networks for Texture Classification*. [S.l.]: Dissertação de Mestrado - Universidade Federal do Paraná, 2014. Citado na página 26.

Haykin, S. *Neural Networks: A Comprehensive Foundation*. Upper Saddle River, NJ: Prentice Hall, 1999. 2nd EDITION. Citado na página 22.

Haykin, S. *Redes Neurais - 2ed.* [S.l.]: Bookman, 2001. ISBN 9788573077186. Citado na página 27.

Hebb, D. O. *The organization of behavior: neuropsychological theory*. New York: Wiley, 1949. Jun. ISBN 0-8058-4300-0. Citado na página 22.

HJELSETH, E. Bim-based model checking (bmc). In: _____. [S.l.]: Building Information Modeling: Applications and Practices, 2015. p. 33–61. ISBN 978-0-7844-1398-2. Citado na página 41.

HJELSETH, E.; NISBET, N. N. Capturing normative constraints by use of the semantic mark-up rase methodology. In: *Capturing normative constraints by use of the semantic mark-up RASE methodology*. [s.n.], 2011. Disponível em: <<https://api.semanticscholar.org/CorpusID:62759547>>. Citado 2 vezes nas páginas 11 e 41.

HOCH. 2016. Citado 2 vezes nas páginas 4 e 15.

Hochreiter, S.; Schmidhuber, J. Long short-term memory. *Neural Comput.*, MIT Press, Cambridge, MA, USA, v. 9, n. 8, p. 1735–1780, nov 1997. ISSN 0899-7667. Citado 2 vezes nas páginas 29 e 30.

IBRAHIM ROBERT KRAWCZYK, G. S. M. Two approaches to bim: A comparative study. *Illinois Institute of Technology*, Chicago, 2004. Citado na página 14.

INAN, H. et al. Llama guard: Llm-based input-output safeguard for human-ai conversations. *ArXiv*, abs/2312.06674, 2023. Disponível em: <<https://api.semanticscholar.org/CorpusID:266174345>>. Citado na página 10.

IZACARD, G.; GRAVE, E. Leveraging passage retrieval with generative models for open domain question answering. In: MERLO, P.; TIEDEMANN, J.; TSARFATY, R. (Ed.). *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. Online: Association for Computational Linguistics, 2021. p. 874–880. Disponível em: <<https://aclanthology.org/2021.eacl-main.74/>>. Citado na página 39.

Jurafsky, D.; Martin, J. H. *Speech and Language Processing (2nd Edition)*. Usa: Prentice-Hall, Inc., 2009. ISBN 0131873210. Citado na página 18.

LEWIS, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. 2021. Disponível em: <<https://arxiv.org/abs/2005.11401>>. Citado na página 39.

- Liddy, E. D. *Enhanced text retrieval using natural language processing*. [S.l.]: American Society for Information Science and Technology, 1998. 14–16 p. Citado 2 vezes nas páginas 18 e 19.
- LIN, W. Y. Prototyping a chatbot for site managers using building information modeling (bim) and natural language understanding (nlu) techniques. *Sensors*, v. 23, n. 6, 2023. ISSN 1424-8220. Disponível em: <<https://www.mdpi.com/1424-8220/23/6/2942>>. Citado 2 vezes nas páginas 44 e 45.
- LIU, Y.; ZHANG, X.; LI, H. Bim machine learning and design rules to improve the assembly process. *Sustainability*, v. 14, n. 1, p. 288, 2022. Disponível em: <<https://www.mdpi.com/2071-1050/14/1/288>>. Citado na página 10.
- Manning, C.; Raghavan, P.; Schütze, H. *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008. ISBN 978-0-521-86571-5. Citado na página 40.
- Martin, J. H.; Jurafsky, D. *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition*. [S.l.]: Pearson/Prentice Hall Upper Saddle River, 2009. Citado na página 21.
- McCaffrey, J. *Why You Should Use Cross-Entropy Error Instead Of Classification Error Or Mean Squared Error For Neural Network Classifier Training*. 2013. Acesso em: 12/02/2021. Citado na página 26.
- Mcculloch, W.; Pitts, W. A logical calculus of ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, p. 127–147, 1943. 5. Citado na página 22.
- MENDONÇA, E. A. d.; MANZIONE, L. *Conversão de regras de acessibilidade pela metodologia Rase para verificação automática em modelo BIM*. 2020. Citado 3 vezes nas páginas 47, 49 e 54.
- Mikolov, T. et al. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013. Disponível em: <<https://arxiv.org/abs/1301.3781>>. Citado na página 40.
- Minsky, M.; Papert, S. *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA, USA: MIT Press, 1969. 355–368 p. Citado na página 23.
- Mitchell, T. M. *Machine Learning*. 1. ed. Nova York: McGraw-Hill International Editions, 1997. 14 p. ISBN 978-00-7115-467-3. Citado na página 15.
- MONTEIRO, A. *Projeto para a produção de vedações verticais em alvenaria em uma ferramenta CAD-BIM*. [S.l.]: Dissertação de Mestrado - Escola Politécnica da Universidade de São Paulo, 2011. Citado na página 13.
- Mun, J.; Cho, M.; Han, B. *Text-guided Attention Model for Image Captioning*. 2016. Citado na página 31.
- NASCIMENTO, J. R. *Exploração de técnicas de engenharia de prompt para aprimorar os resultados do uso de LLM no TCMRio*. Natal: [s.n.], 2024. Acesso em: 25 maio 2024. Disponível em: <<https://repositorio.ufrn.br/handle/123456789/58251>>. Citado na página 40.
- Nasr, G. E.; Badr, E. A.; Joun, C. Cross entropy error function in neural networks: Forecasting gasoline demand. In: *Proceedings of the Fifteenth International Florida Artificial Intelligence Research Society Conference*. [S.l.]: AAAI Press, 2002. p. 381–384. ISBN 157735141x. Citado na página 27.

- NAVEED, H. et al. *A Comprehensive Overview of Large Language Models*. 2024. Disponível em: <<https://arxiv.org/abs/2307.06435>>. Citado na página 37.
- Neves, R. de Cássia David das. *Pré-processamento no processo de descoberta de conhecimento em banco de dados*. [S.l.]: Dissertação de Mestrado - UFRGS Lume, 2003. Citado na página 16.
- OPENAI et al. *GPT-4 Technical Report*. 2024. Disponível em: <<https://arxiv.org/abs/2303.08774>>. Citado 2 vezes nas páginas 10 e 37.
- PARREIRA, J. P. de C. *Implementação BIM nos processos organizacionais em empresas de construção – um caso de estudo*. [S.l.]: Dissertação de Mestrado - Faculdade de Ciências e Tecnologia, 2013. Citado na página 13.
- PENTTILÄ, H. Describing the changes in architectural information technology to understand design complexity and free-form architectural expression. *Journal of Information Technology in Construction*, v. 11, n. 4, 02 2016. Citado na página 13.
- Popel, M.; Bojar, O. Training tips for the transformer model. *The Prague Bulletin of Mathematical Linguistics*, Charles University in Prague, Karolinum Press, v. 110, n. 1, p. 43–70, Apr 2018. ISSN 1804-0462. Citado 2 vezes nas páginas 34 e 35.
- RADFORD, A.; WU, J.; AMODEI, D. Learning transferable visual models from natural language supervision. In: *Proceedings of the International Conference on Machine Learning (ICML)*. [S.l.: s.n.], 2019. v. 32, p. 5049–5059. Citado 2 vezes nas páginas 50 e 51.
- RAFAILOV, R. et al. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. 2024. Disponível em: <<https://arxiv.org/abs/2305.18290>>. Citado na página 37.
- Ramakrishnan, N.; Soni, T. Network traffic prediction using recurrent neural networks. *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, p. 187–193, 2018. Citado 2 vezes nas páginas 28 e 30.
- RANE, N.; CHOUDHARY, S.; RANE, J. Integrating building information modelling (bim) with chatgpt, bard, and similar generative artificial intelligence in the architecture, engineering, and construction industry: applications, a novel framework, challenges, and future scope. *SSRN Electronic Journal*, 01 2023. Citado 2 vezes nas páginas 44 e 45.
- Rezende, S. O.; Marcacini, R. M.; Moura, M. F. O uso da mineração de textos para extração e organização não supervisionada de conhecimento. *Revista de Sistemas de Informação da FSMA*, v. 7, p. 7–21, 2011. ISSN 19835604. Citado na página 20.
- Rojas, R. *Neural Networks - A Systematic Introduction*. Berlin: Springer-Verlag, 1996. 154–155 p. Citado na página 24.
- Rosenblatt, F. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, v. 65, n. 6, p. 386–408, 1958. Citado na página 22.
- Rumelhart, D. E.; Hinton, G. E.; Williams, R. J. Learning Representations by Back-propagating Errors. *Nature*, v. 323, n. 6088, p. 533–536, 1986. Citado na página 27.
- RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 3. ed. [S.l.]: Prentice Hall, 2010. Citado 4 vezes nas páginas 16, 18, 26 e 27.

- SAKA, A. et al. Integrated bim and machine learning system for circularity prediction of construction demolition waste. *arXiv preprint arXiv:2407.14847*, 2024. Disponível em: <<https://arxiv.org/abs/2407.14847>>. Citado na página 10.
- SAMMOUR, F. et al. *Responsible AI in Construction Safety: Systematic Evaluation of Large Language Models and Prompt Engineering*. 2024. Disponível em: <<https://arxiv.org/abs/2411.08320>>. Citado 2 vezes nas páginas 44 e 45.
- SAMSAMI, R. Optimizing the utilization of generative artificial intelligence (ai) in the aec industry: Chatgpt prompt engineering and design. *CivilEng*, v. 5, n. 4, p. 971–1010, 2024. ISSN 2673-4109. Disponível em: <<https://www.mdpi.com/2673-4109/5/4/49>>. Citado 3 vezes nas páginas 45, 46 e 50.
- SANTOS, K. T.; SAMPAIO, M. A. B. Aplicação da metodologia rase na norma de acessibilidade associada ao bim. *SIMPÓSIO BRASILEIRO DE GESTÃO E ECONOMIA DA CONSTRUÇÃO*, v. 12, n. 00, p. 1–8, out. 2021. Disponível em: <<https://eventos.antac.org.br/index.php/sibragec/article/view/500>>. Citado na página 11.
- Santos, R. et al. Técnicas de processamento de linguagem natural aplicadas ao processo de mineração de textos: Resultados preliminares de um mapeamento sistemático. In: *Técnicas de processamento de linguagem natural aplicadas ao processo de mineração de textos: Resultados preliminares de um mapeamento sistemático*. [S.l.: s.n.], 2015. Citado na página 19.
- Schmidhuber, J. Deep learning in neural networks: An overview. *Neural Networks*, Elsevier BV, v. 61, p. 85–117, Jan 2015. ISSN 0893-6080. Citado na página 27.
- Sensoy, M.; Kaplan, L.; Kandemir, M. Evidential deep learning to quantify classification uncertainty. In: *Evidential Deep Learning to Quantify Classification Uncertainty*. [S.l.: s.n.], 2018. p. 3179–3189. Citado na página 26.
- Shao, L.; Zhu, F.; Li, X. Transfer learning for visual categorization: a survey. *IEEE transactions on neural networks and learning systems*, v. 26, n. 5, p. 1019–1034, May 2015. ISSN 2162-237x. Citado na página 18.
- Simard, P.; Steinkraus, D.; Platt, J. Best practices for convolutional neural networks applied to visual document analysis. In: *Seventh International Conference on Document Analysis and Recognition, 2003. Proceedings*. [S.l.: s.n.], 2003. p. 958–963. Citado na página 31.
- Soares, D.; Teive, R. Estudo comparativo entre as redes neurais artificiais mlp e rbf para previsão de cheias em curto prazo. *Revista de Informática Teórica e Aplicada*, v. 22, n. 2, p. 67–86, 2015. ISSN 21752745. Citado na página 23.
- SOUZA, L. L. A. de. *Diagnóstico do uso do BIM em empresas de projeto de Arquitetura*. [S.l.]: Dissertação de Mestrado - Universidade Federal Fluminense, 2009. Citado na página 14.
- STRUBELL, E.; GANESH, A.; MCCALLUM, A. *Energy and Policy Considerations for Deep Learning in NLP*. 2019. Disponível em: <<https://arxiv.org/abs/1906.02243>>. Citado na página 36.
- Sutskever, I.; Vinyals, O.; Le, Q. V. Sequence to sequence learning with neural networks. In: Ghahramani, Z. et al. (Ed.). *Advances in Neural Information Processing Systems*. [S.l.]: Curran Associates, Inc., 2014. v. 27. Citado na página 30.

Tafner, M. A.; Xerez, M. de; Filho, E. R. *Malcon Anderson Tafner, Marcos de Xerez, Eulampio Rodrigues Filho*. Belo Horizonte: Blumenau, 1995. ISBN 8571140502. Citado na página 22.

Tai, K. S.; Socher, R.; Manning, C. Improved semantic representations from tree-structured long short-term memory networks. In: *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Beijing, China: Association for Computational Linguistics, 2015. p. 1556–1566. Citado 2 vezes nas páginas 29 e 30.

TAN, P.-N.; STEINBACH, M.; KUMAR, V. *Introdução a DATAMINING Mineração de Dados*. 2. ed. Rio de Janeiro: Person Education, Inc., 2006. ISBN 978-85-7393-761-9. Citado 3 vezes nas páginas 10, 15 e 16.

TOUVRON, H. et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023. Disponível em: <<https://arxiv.org/abs/2302.13971>>. Citado 2 vezes nas páginas 36 e 37.

VARGAS, F. G. Digitalização na construção: O uso do bim. *Blog do IBRE - FGV*, 2023. Disponível em: <<https://blogdoibre.fgv.br/posts/digitalizacao-na-construcao-o-uso-do-bim>>. Citado na página 9.

VARGAS, J. Análise da produção científica brasileira sobre a modelagem da informação da construção (bim). *Ambiente Construído*, SciELO, v. 15, n. 4, p. 183–201, 2023. Citado na página 10.

Vaswani, A. et al. *Attention Is All You Need*. 2017. Disponível em: <<https://arxiv.org/abs/1706.03762>>. Citado 6 vezes nas páginas 32, 33, 34, 35, 37 e 38.

Vieira, R.; Lima, V. L. S. de. *Linguística computacional: princípios e aplicações*. Jaia, SBC, Fortaleza, Brasil, 2001. Citado na página 19.

Willmott, C. J. On the validation of models. *Physical Geography*, Taylor and Francis, v. 2, n. 2, p. 184–194, 1981. Citado na página 26.

ZHANG, R. et al. *LLaMA-Adapter: Efficient Fine-tuning of Language Models with Zero-init Attention*. 2024. Disponível em: <<https://arxiv.org/abs/2303.16199>>. Citado na página 41.

ZHAO, W. X. et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023. Disponível em: <<https://arxiv.org/abs/2303.18223>>. Citado na página 11.